ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



Họ và tên sinh viên: Đào Duy Hưng

Mã số sinh viên: 3122410158

LAB02 SEARCHING

Môn học : **Trí tuệ nhân tạo nâng cao**

Giảng viên : **ThS. Đỗ Như Tài**

TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1: Cấu trúc cây tìm kiếm và lớp Node.....2
1.1. Mục tiêu bài thực hành.....2
1.2. Cấu trúc lớp Node.....2
1.2.1. Các thuộc tính pos, parent, action, cost.....3
1.2.2. Quan hệ giữa nút cha và nút con.....4
1.3. Đặc điểm của cây tìm kiếm trong AI.....5
1.3.1. Cơ chế hình thành cây tìm kiếm động.....5
1.3.2. Phân biệt cây tìm kiếm và cây nhị phân.....6
1.4. Truy hồi đường đi bằng get_path_from_root().....6
1.5. Kết quả thực nghiệm và nhận xét.....8
CHƯƠNG 2: Mô hình hóa bài toán mê cung.....10
2.1. Giới thiệu bài toán mê cung.....10
2.1.1. Biểu diễn mê cung bằng ma trận ký tự.....10
2.1.2. Quy ước các ô X, S, G và khoảng trắng.....11
2.2. Mô hình hóa bài toán tìm kiếm.....12
2.2.1. Trạng thái bắt đầu và trạng thái đích.....12
2.2.2. Tập hành động và mô hình chuyển trạng thái.....12
2.2.3. Chi phí đường đi.....13
2.3. Phân tích không gian trạng thái.....13
2.3.1. Kích thước không gian trạng thái n.....14
2.3.2. Độ sâu lời giải tối ưu d.....14
2.3.3. Độ sâu tối đa m.....14
2.3.4. Hệ số phân nhánh tối đa b.....15
2.4. Kết quả khảo sát các mê cung.....15
2.5. Nhận xét.....17
CHƯƠNG 3: Tìm kiếm không thông tin: BFS và DFS.....19
3.1. Cơ sở lý thuyết.....19
3.1.1. Thuật toán Breadth-First Search (BFS).....19
3.1.2. Thuật toán Depth-First Search (DFS).....20
3.2. Triển khai thuật toán tìm kiếm.....20
3.2.1. Cấu trúc Queue và Stack.....21
3.2.2. Quản lý frontier và trạng thái mở rộng.....21
3.3. Kết quả thực nghiệm.....22

3.3.1. Kết quả thực nghiệm BFS.....22
3.3.2. Kết quả thực nghiệm DFS qua nhiều lần chạy.....23
3.4. So sánh BFS và DFS.....24
3.5. Nhận xét.....27
CHƯƠNG 4: Depth-Limited DFS và Iterative Deepening Search.....28
4.1. Thuật toán Depth-Limited DFS (DLS).....28
4.1.1. Nguyên lý giới hạn độ sâu.....28
4.1.2. Ảnh hưởng của giá trị giới hạn L.....29
4.2. Thuật toán Iterative Deepening Search (IDS).....29
4.2.1. Nguyên lý tăng dần giới hạn độ sâu.....29
4.2.2. Sự kết hợp các đặc tính của BFS và DFS.....30
4.3. Kết quả thực nghiệm DLS và IDS.....31
4.3.1. Kết quả DLS với các giới hạn khác nhau.....31
4.3.2. Số lần tăng giới hạn độ sâu của IDS.....32
4.4. Ảnh hưởng của cycle checking.....35
4.5. Nhận xét.....36
CHƯƠNG 5: Tìm kiếm có thông tin: GBFS, A* và Weighted A*.....37
5.1. Cơ sở lý thuyết các thuật toán tìm kiếm có thông tin.....37
5.1.1. Greedy Best-First Search (GBFS).....38
5.1.2. Thuật toán A*.....39
5.1.3. Thuật toán Weighted A*.....39
5.2. Hàm đánh giá và heuristic.....40
5.2.1. Các thành phần g(n),h(n),f(n).....40
5.2.2. Khoảng cách Manhattan và Euclidean.....41
5.3. Kết quả thực nghiệm.....42
5.3.1. So sánh GBFS, A* và BFS.....42
5.3.2. Khảo sát Weighted A* với các trọng số w khác nhau.....46
5.4. Đánh giá sự đánh đổi giữa hiệu suất và chất lượng lời giải.....47
5.5. Nhận xét.....49
CHƯƠNG 6: Ảnh hưởng của heuristic.....50
6.1. Vai trò của heuristic trong bài toán tìm kiếm.....50
6.2. Khảo sát giá trị heuristic.....50
6.2.1. Heuristic Manhattan.....50
6.2.2. Heuristic Euclidean.....51

6.2.3. Quan sát bản đồ h(n) và f(n)..... 52
6.3. So sánh hiệu quả heuristic đối với GBFS và A*.....54
6.4. Đề xuất heuristic thay thế.....57
6.4.1. Heuristic Chebyshev..... 57
6.4.2. Điều kiện admissibility và consistency.....58
6.5. Nhận xét..... 59
CHƯƠNG 7: Kiểm tra chu trình trong DFS..... 61
7.1. Vấn đề chu trình trong tìm kiếm theo chiều sâu..... 61
7.2. Các phương án kiểm tra chu trình.....61
7.2.1. Kiểm tra trạng thái trên đường đi hiện tại.....61
7.2.2. Kiểm tra đường đi và tránh thêm trạng thái đã có trong frontier.....62
7.2.3. Kiểm tra đường đi và điều chỉnh thứ tự trong frontier.....63
7.3. Thực nghiệm trên open_maze.....63
7.4. So sánh các phương án cycle checking.....65
7.4.1. Số lần lặp và số nút mở rộng.....65
7.4.2. Kích thước frontier.....66
7.4.3. Hiện tượng lặp trạng thái..... 68
7.5. Nhận xét..... 69
CHƯƠNG 8: Bài tập tổng hợp – Goal-based Planning Agent.....71
8.1. Giới thiệu Goal-based Planning Agent..... 71
8.1.1. Mục tiêu và chức năng của tác tử.....71
8.1.2. Quy trình từ trạng thái đầu đến trạng thái đích..... 72
8.2. Thiết kế và triển khai chương trình.....73
8.2.1. Biểu diễn trạng thái và lớp Node..... 73
8.2.2. Tổ chức cây tìm kiếm, frontier và reached..... 73
8.2.3. Triển khai BFS, DFS, GBFS và A*..... 74
8.3. Phần thực hành nâng cao.....75
8.3.1. Iterative Deepening Search (IDS)..... 75
8.3.2. Bài toán tìm kiếm nhiều đích..... 76
8.4. Tổng hợp kết quả thực nghiệm..... 77
8.4.1. Môi trường và điều kiện thực nghiệm.....77
8.4.2. Bảng so sánh các thuật toán trên các mê cung.....77
8.4.3. Trực quan hóa đường đi và trạng thái tìm kiếm.....79
8.5. Đánh giá tổng hợp các thuật toán tìm kiếm..... 82

8.5.1. Tính đầy đủ và tính tối ưu..... 82
8.5.2. Thời gian thực thi và nhu cầu bộ nhớ..... 82
8.5.3. Vai trò của heuristic..... 83
8.5.4. Ảnh hưởng của cycle checking..... 84
8.6. Kết luận bài thực hành tổng hợp..... 84

LỜI MỞ ĐẦU

Trong bối cảnh trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong học tập, nghiên cứu và thực tiễn, việc hiểu rõ các nguyên lý, phương pháp và cách xây dựng hệ thống trí tuệ nhân tạo là nền tảng cần thiết đối với sinh viên theo định hướng công nghệ. Môn học Trí tuệ nhân tạo nâng cao giúp người học tiếp cận các kiến thức và vấn đề cốt lõi của lĩnh vực AI, đồng thời rèn luyện khả năng phân tích, tư duy logic và vận dụng kiến thức vào những bài toán thực tế.

Assignment này được thực hiện nhằm củng cố và hệ thống hóa những kiến thức đã được học thông qua việc nghiên cứu, phân tích và trả lời các câu hỏi theo từng chủ đề. Thay vì chỉ tiếp cận kiến thức ở mức lý thuyết, quá trình thực hiện bài tập giúp tôi hiểu rõ hơn cách các khái niệm AI được hình thành, cách chúng liên kết với nhau và cách có thể áp dụng vào các bài toán cụ thể.

Trong quá trình thực hiện, tôi tập trung vào việc tự tìm hiểu nội dung, đối chiếu với tài liệu môn học, phân tích vấn đề và trình bày kết quả theo hướng rõ ràng, có lập luận. Qua đó, tôi không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện khả năng nghiên cứu tài liệu, giải quyết vấn đề và đánh giá một hệ thống AI dưới nhiều góc độ khác nhau.

Thông qua assignment này và các bài tập tiếp theo, tôi kỳ vọng có thể xây dựng nền tảng kiến thức vững chắc hơn về Trí tuệ nhân tạo, đồng thời từng bước nâng cao khả năng vận dụng kiến thức vào các bài toán thực tế. Đây cũng là cơ sở để tôi tiếp tục phát triển các kỹ năng chuyên môn và nghiên cứu sâu hơn trong lĩnh vực AI trong quá trình học tập và định hướng nghề nghiệp sau

CHƯƠNG 1: Cấu trúc cây tìm kiếm và lớp Node

1.1. Mục tiêu bài thực hành

Bài thực hành nhằm tìm hiểu cấu trúc cây tìm kiếm (Search Tree) trong trí tuệ nhân tạo và cách biểu diễn các nút tìm kiếm bằng ngôn ngữ lập trình Python. Đây là một trong những cấu trúc dữ liệu cơ bản được sử dụng trong các bài toán giải quyết vấn đề bằng tìm kiếm, cho phép biểu diễn trạng thái, hành động và mối quan hệ giữa các trạng thái trong quá trình tìm kiếm lời giải.

Thông qua notebook 01_trees.ipynb, bài thực hành tập trung nghiên cứu lớp Node với bốn thuộc tính pos, parent, action và cost; phân tích cơ chế hình thành cây tìm kiếm động và sự khác biệt giữa cây tìm kiếm trong AI với cây nhị phân thông thường. Bên cạnh đó, bài thực hành sử dụng phương thức get_path_from_root() để truy hồi dãy trạng thái và hành động từ nút hiện tại về nút gốc.

Kết quả của bài thực hành giúp làm rõ cách tổ chức và quản lý các nút trong cây tìm kiếm, đồng thời tạo cơ sở để triển khai các thuật toán tìm kiếm như Breadth-First Search (BFS), Depth-First Search (DFS) và A* trong các bài thực hành tiếp theo.

1.2. Cấu trúc lớp Node

Trong cây tìm kiếm AI, mỗi nút biểu diễn một trạng thái được tạo ra trong quá trình tìm kiếm. Nút không chỉ chứa thông tin về trạng thái hiện tại mà còn lưu trữ mối quan hệ với nút cha, hành động đã thực hiện và chi phí tích lũy từ trạng thái bắt đầu.

Trong notebook, lớp Node được xây dựng bằng Python với phương thức khởi tạo __init__(self, pos, parent, action, cost). Ngoài các thuộc tính cơ bản, lớp còn cung cấp phương thức __str__() để hiển thị thông tin nút và get_path_from_root() để truy hồi đường đi.

1.2.1. Các thuộc tính pos, parent, action, cost

Lớp Node sử dụng bốn thuộc tính chính để biểu diễn một nút trong cây tìm kiếm. Ý nghĩa của từng thuộc tính được trình bày trong Bảng 1.1.

Bảng 1.1. Các thuộc tính của lớp Node

Thuộc tính	Ý nghĩa	Ví dụ
pos	Biểu diễn trạng thái hiện tại của nút bằng tọa độ hàng và cột (row, col).	(2, 3)
parent	Lưu tham chiếu đến nút cha đã sinh ra nút hiện tại. Đối với nút gốc, giá trị bằng None.	root_demo
action	Biểu diễn hành động được thực hiện từ nút cha để chuyển đến nút hiện tại.	"E" – Di chuyển sang phải
cost	Lưu tổng chi phí tích lũy từ nút gốc đến nút hiện tại.	1

Thuộc tính pos xác định vị trí hoặc trạng thái mà nút đại diện. Trong bài toán mê cung, trạng thái được biểu diễn dưới dạng bộ tọa độ (row, col). Việc lưu trạng thái bằng kiểu dữ liệu tuple giúp thống nhất cách biểu diễn vị trí trong quá trình tìm kiếm.

Thuộc tính parent đóng vai trò liên kết giữa các nút trong cây. Thông qua thuộc tính này, một nút có thể truy cập trực tiếp đến nút cha, từ đó lần ngược lại các trạng thái trước đó mà không cần lưu trữ toàn bộ đường đi trong từng nút.

Thuộc tính action cho biết hành động đã dẫn đến trạng thái hiện tại. Trong ví dụ thực hành, các hành động được ký hiệu bằng các chữ cái chỉ hướng di chuyển, chẳng hạn N (North – lên), E (East – phải) và S (South – xuống).

Thuộc tính cost biểu diễn tổng chi phí của đường đi từ nút gốc đến nút hiện tại. Khi mỗi bước di chuyển có chi phí bằng 1, giá trị này tương ứng với số bước đã thực hiện. Trong thuật toán A*, chi phí tích lũy cũng được biểu diễn bằng hàm g(n).

Đối với nút gốc, hai thuộc tính parent và action đều nhận giá trị None, trong khi cost = 0 do chưa thực hiện bất kỳ hành động di chuyển nào.

1.2.2. Quan hệ giữa nút cha và nút con

Quan hệ giữa các nút được thiết lập thông qua thuộc tính parent. Mỗi nút con lưu tham chiếu đến nút đã sinh ra nó, qua đó hình thành cấu trúc phân cấp từ nút gốc đến các nút ở những độ sâu khác nhau.

Trong ví dụ ban đầu của notebook, nút gốc được khởi tạo tại vị trí (0, 0) với chi phí bằng 0. Sau đó, nút node2 được tạo tại vị trí (0, 1) thông qua hành động E, có parent tham chiếu đến nút gốc và chi phí tích lũy bằng 1.

Điều này thể hiện mối quan hệ giữa hai nút: trạng thái (0, 1) được sinh ra từ trạng thái (0, 0) sau một bước di chuyển sang phải.

Notebook còn minh họa cơ chế quản lý tham chiếu trong Python thông qua hàm sys.getrefcount(). Khi biến root và thuộc tính node2.parent cùng tham chiếu đến nút gốc, đối tượng có nhiều tham chiếu hơn. Sau khi gán root = None, nút gốc vẫn có thể được truy cập thông qua node2.parent.

Cơ chế này cho thấy việc duy trì liên kết từ nút con đến nút cha giúp bảo toàn thông tin cần thiết để truy hồi đường đi. Một đối tượng chỉ có thể được thu hồi khi không còn các tham chiếu giữ nó tồn tại. Điều này cũng giải thích tại sao việc quản lý các nút và tham chiếu có vai trò quan trọng đối với bộ nhớ trong các thuật toán tìm kiếm.

1.3. Đặc điểm của cây tìm kiếm trong AI

Cây tìm kiếm là cấu trúc dùng để biểu diễn các khả năng phát triển trạng thái trong quá trình giải quyết bài toán. Khác với một số cấu trúc cây dữ liệu truyền thống, cây tìm kiếm trong AI thường được xây dựng theo nhu cầu của thuật toán và số lượng nút con phụ thuộc vào các hành động có thể thực hiện tại từng trạng thái.

1.3.1. Cơ chế hình thành cây tìm kiếm động

Cây tìm kiếm trong AI thường được xây dựng động (Dynamic Search Tree), nghĩa là các nút được tạo ra trong quá trình thuật toán tìm kiếm thay vì khởi tạo trước toàn bộ cây.

Ban đầu, cây chỉ chứa nút gốc đại diện cho trạng thái bắt đầu. Khi thuật toán lựa chọn một nút để mở rộng, các hành động hợp lệ tại trạng thái đó sẽ được xem xét nhằm tạo ra những nút con tương ứng. Quá trình tiếp tục diễn ra cho đến khi tìm thấy trạng thái đích hoặc thỏa mãn điều kiện dừng của thuật toán.

Cách xây dựng này giúp thuật toán không cần lưu trữ trước toàn bộ các trạng thái và đường đi có thể có. Trong một số chiến lược tìm kiếm, chẳng hạn DFS, những nhánh không còn cần thiết có thể được loại bỏ khỏi cấu trúc đang quản lý để giảm nhu cầu sử dụng bộ nhớ, với điều kiện không còn tham chiếu khác giữ lại các nút đó.

Trong phần thực nghiệm bổ sung, một cây minh họa được tạo bằng cách khởi tạo lần lượt các nút từ root_demo đến node_ees. Mỗi nút con được gắn với một nút cha cụ thể, cho thấy cấu trúc cây có thể được mở rộng từng bước mà không cần xác định trước toàn bộ số lượng nút.

Mặc dù các nút trong ví dụ được khởi tạo trực tiếp bằng Python, cách tổ chức này phản ánh cơ chế tạo nút được sử dụng trong các thuật toán tìm kiếm tự động.

1.3.2. Phân biệt cây tìm kiếm và cây nhị phân

Cây nhị phân (Binary Tree) là cấu trúc dữ liệu mà mỗi nút có tối đa hai nút con, thường được gọi là nút con trái và nút con phải. Trong khi đó, cây tìm kiếm AI không bị giới hạn bởi số lượng hai nút con tại mỗi trạng thái.

Số lượng nút con trong cây tìm kiếm AI phụ thuộc vào các hành động hợp lệ của bài toán. Chẳng hạn, đối với bài toán tìm đường trong mê cung cho phép di chuyển theo bốn hướng, một trạng thái có thể sinh ra tối đa bốn nút con tương ứng với các hướng lên, xuống, trái và phải, trước khi xét đến các giới hạn như tường hoặc biên bản đồ.

Trong ví dụ thực nghiệm, nút gốc root_demo tại vị trí (2, 2) được mở rộng thành ba nút con thông qua các hành động N, E và S. Các nút con lần lượt biểu diễn những trạng thái (1, 2), (2, 3) và (3, 2).

Như vậy, chỉ riêng nút gốc đã có ba nút con, cho thấy cây tìm kiếm được xây dựng trong ví dụ không phải là cây nhị phân.

Ngoài ra, lớp Node trong notebook không lưu trực tiếp danh sách các nút con. Thay vào đó, mỗi nút con lưu một tham chiếu đến nút cha thông qua thuộc tính parent. Cách biểu diễn này phù hợp với yêu cầu truy hồi đường đi trong các bài toán tìm kiếm, bởi sau khi xác định được nút đích, thuật toán chỉ cần lần theo các liên kết cha để khôi phục lời giải.

1.4. Truy hồi đường đi bằng get_path_from_root()

Trong các bài toán tìm kiếm, việc xác định trạng thái đích chưa đủ để hoàn thành lời giải. Thuật toán còn cần khôi phục chuỗi trạng thái và hành động đã thực hiện từ trạng thái ban đầu đến trạng thái đích.

Để thực hiện nhiệm vụ này, lớp Node cung cấp phương thức get_path_from_root(). Phương thức sử dụng các liên kết parent để truy hồi đường đi từ nút hiện tại về nút gốc.

Quá trình truy hồi bắt đầu bằng việc đưa nút hiện tại vào danh sách path. Sau đó, chương trình liên tục truy cập thuộc tính parent để chuyển đến nút cha và bổ sung nút đó vào danh sách. Quá trình kết thúc khi gặp nút gốc, tức là nút có parent = None.

Do các nút được thu thập theo thứ tự từ nút hiện tại về nút gốc, phương thức sử dụng path.reverse() để đảo ngược danh sách, từ đó thu được đường đi theo đúng chiều từ trạng thái bắt đầu đến trạng thái hiện tại.

Giá trị trả về của phương thức là danh sách các đối tượng Node, không phải trực tiếp danh sách tọa độ hoặc hành động. Từ danh sách này, chương trình có thể trích xuất các thuộc tính pos, action và cost để biểu diễn lời giải theo từng mục đích khác nhau.

Trong phần thực nghiệm bổ sung, phương thức được gọi tại nút node_ees, có vị trí (3, 4). Kết quả truy hồi như sau:

Dãy trạng thái: (2, 2) → (2, 3) → (2, 4) → (3, 4)

Dãy hành động: E → E → S

Chi phí tích lũy: 0 → 1 → 2 → 3

Dãy trạng thái cho biết thứ tự các vị trí được đi qua, trong khi dãy hành động biểu diễn những bước di chuyển tương ứng. Giá trị None tại nút gốc được loại bỏ khi trích xuất chuỗi hành động vì không có hành động nào được thực hiện để đi đến trạng thái bắt đầu.

Với giả định mỗi bước di chuyển có chi phí bằng 1, đường đi gồm ba bước và có tổng chi phí bằng 3. Tổng chi phí được xác định bằng giá trị cost của nút cuối cùng trên đường đi.

Như vậy, phương thức get_path_from_root() cho phép khôi phục đầy đủ lời giải từ các liên kết giữa nút cha và nút con mà không cần lưu riêng toàn bộ đường đi trong mỗi nút.

1.5. Kết quả thực nghiệm và nhận xét

Sau khi thực hiện các cell trong notebook 01_trees.ipynb, chương trình đã minh họa được quá trình khởi tạo nút, thiết lập quan hệ cha – con và truy hồi đường đi trong cây tìm kiếm.

Để quan sát rõ hơn các thuộc tính của lớp Node, phần thực nghiệm bổ sung xây dựng một cây gồm sáu nút, với nút gốc tại vị trí (2, 2) và các nhánh được mở rộng theo những hướng di chuyển khác nhau.

Kết quả thu được được tổng hợp trong Bảng 1.2.

Bảng 1.2. Kết quả thực nghiệm các thuộc tính của lớp Node

Nút	pos	parent.pos	action	cost
root_demo	(2, 2)	None	None	0
node_n	(1, 2)	(2, 2)	N	1
node_e	(2, 3)	(2, 2)	E	1
node_s	(3, 2)	(2, 2)	S	1
node_ee	(2, 4)	(2, 3)	E	2
node_ees	(3, 4)	(2, 4)	S	3

Từ kết quả thực nghiệm, có thể nhận thấy nút gốc không có nút cha và không gắn với hành động di chuyển. Ba nút node_n, node_e và node_s đều được sinh ra trực tiếp từ nút gốc nên có cùng chi phí tích lũy bằng 1, mặc dù biểu diễn những trạng thái khác nhau.

Đối với nhánh root_demo → node_e → node_ee → node_ees, mỗi nút được tạo ra bằng cách thực hiện thêm một hành động từ nút cha. Do chi phí mỗi bước được quy định bằng 1, giá trị cost tăng lần lượt từ 0 đến 3, phù hợp với số bước di chuyển trên đường đi.

Kết quả cũng cho thấy sự khác biệt giữa vị trí của nút trong cây và trạng thái mà nút biểu diễn. Thuộc tính pos lưu vị trí hiện tại, còn parent và action cung cấp thông tin về cách trạng thái đó được tạo ra. Nhờ đó, chương trình không chỉ xác định được vị trí hiện tại mà còn có thể tái tạo quá trình di chuyển từ trạng thái bắt đầu.

Khi thực hiện phương thức get_path_from_root() tại nút node_ees, chương trình trả về bốn trạng thái theo đúng thứ tự từ nút gốc đến nút hiện tại. Chuỗi hành động tương ứng gồm ba bước E → E → S, với tổng chi phí bằng 3. Các phép kiểm tra được bổ sung trong notebook đều cho kết quả đạt, xác nhận rằng dãy trạng thái, dãy hành động và chi phí tích lũy phù hợp với cấu trúc cây đã xây dựng.

Bên cạnh đó, phần minh họa quản lý tham chiếu trong notebook gốc cho thấy liên kết parent vẫn duy trì khả năng truy cập nút cha ngay cả khi biến ban đầu trỏ đến nút đó được gán bằng None. Điều này thể hiện vai trò của các tham chiếu trong việc duy trì cấu trúc cây và truy hồi thông tin tìm kiếm.

Từ các kết quả trên, có thể kết luận rằng lớp Node cung cấp cấu trúc biểu diễn phù hợp cho cây tìm kiếm AI nhờ khả năng lưu trữ trạng thái, hành động, chi phí tích lũy và quan hệ giữa các nút. Cây tìm kiếm có thể được mở rộng theo các hành động hợp lệ, không bị giới hạn bởi cấu trúc nhị phân và cho phép khôi phục lời giải thông qua liên kết parent.

Những đặc điểm này tạo nền tảng cho việc tổ chức cây tìm kiếm và quản lý đường đi trong các thuật toán BFS, DFS, GBFS và A* được nghiên cứu ở các bài thực hành tiếp theo.

CHƯƠNG 2: Mô hình hóa bài toán mê cung

2.1. Giới thiệu bài toán mê cung

Bài toán mê cung được sử dụng trong Lab 02 nhằm minh họa cách chuyển một bài toán tìm đường thành bài toán tìm kiếm trong không gian trạng thái. Tác tử xuất phát từ một vị trí xác định và cần tìm một dãy hành động hợp lệ để đi đến vị trí đích. Trong quá trình di chuyển, tác tử phải tuân thủ giới hạn của bản đồ và không được đi qua các ô tường.

Dữ liệu thực hành gồm tám mê cung: small, medium, large, open, L, loops, empty và empty_maze_2. Các mê cung có kích thước, cách bố trí tường và mức độ liên thông khác nhau. Notebook Ex3_Show_all_mazes.ipynb được sử dụng để đọc và hiển thị các mê cung, đồng thời các phần mã bổ sung hỗ trợ xác định trạng thái, kiểm tra hành động và tính những đại lượng mô tả không gian tìm kiếm.

2.1.1. Biểu diễn mê cung bằng ma trận ký tự

Mỗi mê cung được lưu dưới dạng tệp văn bản có phần mở rộng .txt. Mỗi dòng trong tệp biểu diễn một hàng, mỗi ký tự biểu diễn một ô của bản đồ. Khi đọc dữ liệu, hàm parse_maze() trong mô-đun maze_helper chuyển nội dung văn bản thành mảng hai chiều NumPy, nhờ đó có thể truy cập một ô bằng chỉ số hàng và cột.

Một trạng thái được mô tả bằng cặp tọa độ (r, c), trong đó r là chỉ số hàng và c là chỉ số cột. Theo quy ước đánh chỉ số của Python, vị trí góc trên bên trái của ma trận là (0, 0); chỉ số hàng tăng theo chiều từ trên xuống dưới, còn chỉ số cột tăng từ trái sang phải. Chẳng hạn, trên mê cung small, bản đồ có 10 hàng và 22 cột. Cách biểu diễn này cho phép xác định vị trí bắt đầu, vị trí đích và các ô lân cận bằng những thao tác trực tiếp trên ma trận.

Bên cạnh việc đọc dữ liệu, notebook sử dụng hàm show_maze() để trực quan hóa cấu trúc bản đồ. Hình ảnh hiển thị giúp quan sát phân bố tường, các vùng có thể di chuyển và vị trí tương đối giữa điểm bắt đầu với đích. Phần thực nghiệm xác định

các đại lượng n, d, m và b trên dữ liệu ma trận, thay vì ước lượng chỉ bằng quan sát hình ảnh.

2.1.2. Quy ước các ô X, S, G và khoảng trắng

Các tệp mê cung sử dụng một tập ký hiệu thống nhất. Ý nghĩa của từng ký hiệu được tổng hợp trong Bảng 2.1.

Bảng 2.1. Quy ước ký hiệu trong dữ liệu mê cung

Ký hiệu	Ý nghĩa	Vai trò trong tìm kiếm
X	Ô tường	Không được phép di chuyển vào
S	Điểm bắt đầu	Xác định trạng thái ban đầu
G	Điểm đích	Xác định trạng thái cần đạt được
Khoảng trắng	Ô trống	Có thể di chuyển qua

Như vậy, các ô mang ký hiệu S, G và khoảng trắng đều là vị trí hợp lệ; các ô X bị loại khỏi không gian trạng thái. Trong quá trình trực quan hóa lời giải, chương trình còn có thể sử dụng những ký hiệu phụ như P để đánh dấu đường đi. Các ký hiệu phụ này phục vụ việc hiển thị kết quả, không thay đổi ý nghĩa của dữ liệu mê cung ban đầu.

2.2. Mô hình hóa bài toán tìm kiếm

Bài toán được biểu diễn bằng năm thành phần: trạng thái bắt đầu, tập hành động, mô hình chuyển trạng thái, trạng thái đích và chi phí đường đi. Việc xác định rõ các thành phần này giúp xây dựng những thuật toán tìm kiếm có cùng dữ liệu đầu vào và quy tắc di chuyển, qua đó bảo đảm tính nhất quán khi so sánh kết quả thực nghiệm.

2.2.1. Trạng thái bắt đầu và trạng thái đích

Trạng thái bắt đầu, ký hiệu là $s_0 = (r□, c□)$, tương ứng với vị trí có ký hiệu S trên bản đồ. Trạng thái đích, ký hiệu là $s^g = (r^g, c^g)$, tương ứng với vị trí có ký hiệu G. Trong Python, hàm find_pos() của maze_helper hỗ trợ xác định hai vị trí này từ mảng ký tự.

Đối với mê cung small, kết quả đọc dữ liệu cho thấy trạng thái bắt đầu là S = (3, 11), còn trạng thái đích là G = (8, 1). Tác tử hoàn thành nhiệm vụ khi trạng thái hiện tại trùng với tọa độ đích, tức GoalTest(s) đúng khi $s = s^g$.

Không gian trạng thái được xác định là tập hợp tất cả các tọa độ thuộc ô hợp lệ trên bản đồ. Cần phân biệt tập hợp này với các trạng thái thực sự có thể đi đến từ S: nếu mê cung có những vùng bị ngăn cách hoàn toàn bởi tường, hai tập hợp có thể khác nhau. Trong tám tệp dữ liệu đang khảo sát, phép kiểm tra của notebook cho thấy toàn bộ các ô hợp lệ đều có thể đi đến từ S.

2.2.2. Tập hành động và mô hình chuyển trạng thái

Trong phạm vi bài thực hành, tác tử chỉ được di chuyển theo bốn hướng: Up (lên), Down (xuống), Left (trái) và Right (phải). Mỗi hành động tương ứng với một độ thay đổi tọa độ: Up = (−1, 0), Down = (1, 0), Left = (0, −1), Right = (0, 1).

Mô hình chuyển trạng thái được biểu diễn bởi $T((r, c), a) = (r + \Delta r_a, c + \Delta c_a)$, với Δr_a và Δc_a là độ thay đổi hàng và cột do hành động a tạo ra. Một phép chuyển chỉ hợp lệ khi vị trí tiếp theo nằm trong giới hạn ma trận và không phải ô tường. Nếu

không thỏa mãn điều kiện này, chương trình không tạo trạng thái kế tiếp cho hành động tương ứng.

Trong notebook, hàm valid_state() kiểm tra tính hợp lệ của vị trí, hàm transition() xác định trạng thái sau khi thực hiện hành động và hàm successors() tạo danh sách các cặp hành động – trạng thái kế tiếp. Cách tổ chức này tách quy tắc môi trường khỏi thuật toán tìm kiếm, giúp các thuật toán sử dụng chung một mô hình chuyển trạng thái.

2.2.3. Chi phí đường đi

Chi phí đường đi biểu diễn tổng chi phí của các hành động được thực hiện từ trạng thái bắt đầu đến một trạng thái đang xét. Nếu đường đi P gồm k hành động, tổng chi phí được xác định bằng $g(P) = c(a_1) + c(a_2) + \ldots + c(a_k)$.

Bài thực hành quy định chi phí của mỗi bước di chuyển hợp lệ bằng 1. Vì vậy, $g(P) = k$, tức chi phí đường đi bằng số bước đã thực hiện. Với quy tắc này, mục tiêu tìm đường đi có chi phí nhỏ nhất tương đương với tìm đường đi có ít bước nhất.

Chẳng hạn, thuật toán BFS xác định đường đi ngắn nhất từ S đến G trên mê cung small có 19 bước, nên chi phí tối ưu tương ứng bằng 19. Quy ước chi phí đồng nhất cũng là cơ sở để sử dụng BFS tính độ sâu lời giải tối ưu trong phần phân tích tiếp theo.

2.3. Phân tích không gian trạng thái

Bốn đại lượng n, d, m và b được sử dụng để mô tả quy mô và cấu trúc của bài toán tìm kiếm. Những đại lượng này phản ánh các khía cạnh khác nhau: số trạng thái có thể đứng, khoảng cách ngắn nhất đến đích, giới hạn độ sâu có thể khảo sát và số lựa chọn di chuyển tại mỗi trạng thái.

2.3.1. Kích thước không gian trạng thái n

Đại lượng n là tổng số ô hợp lệ trên toàn bộ bản đồ, gồm ô trống, điểm bắt đầu và điểm đích. Giá trị này được tính bằng cách đếm tất cả phần tử khác X trong ma trận ký tự. Do đó, n phản ánh quy mô không gian trạng thái theo cách biểu diễn vị trí của bài toán.

Ví dụ, mê cung small có 94 trạng thái hợp lệ, trong khi open có 684 trạng thái hợp lệ. Giá trị n lớn thường làm tăng số trạng thái mà thuật toán có thể phải quản lý, nhưng không trực tiếp quyết định độ dài đường đi tối ưu hay số nút phải mở rộng trong một lần tìm kiếm.

2.3.2. Độ sâu lời giải tối ưu d

Đại lượng d là độ sâu của lời giải nông nhất, được tính bằng số cạnh trên đường đi ngắn nhất từ trạng thái bắt đầu đến trạng thái đích. Vì chi phí mỗi bước di chuyển đều bằng 1, số cạnh trên đường đi cũng chính là chi phí đường đi tối ưu.

Trong notebook, d được xác định bằng Breadth-First Search (BFS). Thuật toán mở rộng trạng thái theo thứ tự độ sâu tăng dần và sử dụng cơ chế ghi nhận trạng thái đã phát hiện để tránh xử lý lại cùng một vị trí. Kết quả cho thấy d = 19 đối với small, d = 68 đối với medium và d = 210 đối với large. Các giá trị này phản ánh số bước ngắn nhất mà tác tử cần thực hiện, không phụ thuộc vào khoảng cách thẳng giữa hai điểm khi các tường cản trở việc di chuyển trực tiếp.

2.3.3. Độ sâu tối đa m

Đại lượng m biểu diễn độ sâu tối đa của cây tìm kiếm, nhưng cách xác định phụ thuộc vào quy tắc xử lý trạng thái lặp. Nếu cho phép quay lại các trạng thái trước đó mà không giới hạn số lần lặp, cây tìm kiếm của một mê cung có chu trình có thể phát triển đến độ sâu không giới hạn. Vì vậy, không thể mặc định m bằng số ô hợp lệ trừ đi 1 trong mọi trường hợp.

Trong phần thực nghiệm, m được khảo sát dưới giả định chỉ xét các đường đi không lặp trạng thái. Với quy tắc này, độ sâu của một đường đi không thể vượt quá

số trạng thái có thể đi đến từ S trừ đi 1. Notebook ghi nhận hai đại lượng phục vụ phân tích: m_DFS là độ sâu lớn nhất quan sát được trong một lần duyệt DFS với thứ tự hướng cố định Up, Down, Left, Right và tập trạng thái đã thăm; m_upper là cận trên lý thuyết bằng số trạng thái có thể tiếp cận từ S trừ đi 1.

Theo đó, có thể biểu diễn $m_DFS \leq m \leq m_upper$ trong phạm vi các đường đi không lặp trạng thái. Giá trị m_DFS chỉ là kết quả quan sát từ một cây DFS, không phải độ sâu lớn nhất chính xác của mọi đường đi hợp lệ. Đối với bộ dữ liệu này, tất cả trạng thái hợp lệ đều có thể đi đến từ S nên $m_upper = n - 1$.

2.3.4. Hệ số phân nhánh tối đa *b*

Đại lượng b là số hành động hợp lệ lớn nhất có thể thực hiện từ một trạng thái. Vì tác tử chỉ có thể di chuyển theo bốn hướng, giá trị b không vượt quá 4. Tuy nhiên, số hành động hợp lệ tại mỗi vị trí còn phụ thuộc vào các ô tường và giới hạn của bản đồ.

Trong notebook, b được tính bằng cách xác định số trạng thái kế tiếp hợp lệ tại từng ô có thể đứng, sau đó lấy giá trị lớn nhất. Kết quả cho thấy small và medium có $b = 3$, trong khi sáu mê cung còn lại có $b = 4$. Hệ số phân nhánh lớn có thể làm tăng số lựa chọn cần xem xét khi mở rộng nút, nhưng số nút thực tế phải khảo sát còn phụ thuộc vào cấu trúc đường đi và chiến lược tìm kiếm.

2.4. Kết quả khảo sát các mê cung

Sau khi chạy các cell bổ sung trong Ex3_Show_all_mazes.ipynb, chương trình đã đọc dữ liệu của tám mê cung, xác định các trạng thái hợp lệ và áp dụng BFS để tính d. Đồng thời, DFS được sử dụng để ghi nhận độ sâu lớn nhất quan sát được, còn hệ số phân nhánh tối đa được tính từ tập các hành động hợp lệ của từng trạng thái.

Kết quả tổng hợp được trình bày trong Bảng 2.2.

Bảng 2.2. Tổng hợp các đại lượng n, d, m và b trên tám mê cung

Mê cung	n	d	m_DFS	m_upper	b
small	94	19	48	93	3
medium	274	68	164	273	3
large	647	210	222	646	4
open	684	54	476	683	4
L	157	16	111	156	4
loops	72	23	42	71	4
empty	100	14	99	99	4
empty_maze_2	100	14	81	99	4

Ghi chú: n là số ô hợp lệ; d là số bước ngắn nhất từ S đến G; m_DFS là độ sâu lớn nhất ghi nhận trong một lượt DFS; m_upper là cận trên khi không lặp trạng thái; b là hệ số phân nhánh tối đa. Hai cột m_DFS và m_upper không được hiểu là hai giá trị chính xác của m.

Từ bảng kết quả, open là mê cung có nhiều trạng thái hợp lệ nhất với n = 684, trong khi large có ít hơn, n = 647. Tuy nhiên, độ sâu lời giải tối ưu trên open chỉ bằng 54, thấp hơn đáng kể so với 210 trên large. Sự khác biệt này cho thấy kích thước không

gian trạng thái không đủ để phản ánh mức độ phức tạp của việc tìm đường đến đích.

Đối với hai mê cung empty và empty_maze_2, giá trị n và d đều lần lượt bằng 100 và 14. Do cả hai đều không có tường bên trong và chỉ cho phép di chuyển bốn hướng, đường đi ngắn nhất được xác định trực tiếp từ độ chênh lệch hàng và cột giữa S và G. Dù vậy, giá trị m_DFS của hai mê cung khác nhau (99 và 81), vì vị trí S, G và thứ tự duyệt ảnh hưởng đến độ sâu của cây DFS được hình thành trong lần chạy cụ thể.

Trên mê cung small, chương trình còn minh họa đường đi ngắn nhất từ S = (3, 11) đến G = (8, 1). Đường đi được hiển thị trên bản sao của ma trận bằng các ô đánh dấu, trong khi dữ liệu bản đồ ban đầu được giữ nguyên. Các phép kiểm tra bổ sung xác nhận đường đi bắt đầu tại S, kết thúc tại G, không đi qua tường và mỗi bước chỉ thay đổi một đơn vị theo hàng hoặc cột.

2.5. Nhận xét

Kết quả khảo sát cho thấy độ khó của bài toán mê cung không chỉ phụ thuộc vào kích thước bản đồ hay tổng số trạng thái hợp lệ mà còn phụ thuộc vào sự phân bố của tường, vị trí bắt đầu và đích, số nhánh rẽ cùng mức độ liên thông giữa các vùng có thể di chuyển.

So sánh open và large cho thấy tác động rõ rệt của cấu trúc môi trường. Mê cung open có 684 trạng thái hợp lệ và độ sâu lời giải tối ưu bằng 54, trong khi large chỉ có 647 trạng thái hợp lệ nhưng cần đến 210 bước để đi từ S đến G. Điều đó cho thấy các lối đi bị chia cắt, hành lang dài và những đoạn phải đi vòng có thể kéo dài đáng kể đường đi ngắn nhất, ngay cả khi số trạng thái không lớn hơn.

Ở các mê cung có nhiều nhánh hoặc tồn tại chu trình, một vị trí có thể được tiếp cận qua nhiều đường đi khác nhau. Nếu không sử dụng cơ chế kiểm tra trạng thái lặp, thuật toán có nguy cơ quay lại các vị trí đã khảo sát, làm gia tăng số nút được tạo ra và chi phí tính toán. Ngược lại, những mê cung có ít ngã rẽ thường giới hạn số hướng di chuyển tại từng trạng thái, mặc dù vẫn có thể buộc tác tử đi qua một hành lang dài để đến đích.

Ngoài ra, kết quả xác định m cho thấy cần phân biệt giữa độ sâu lớn nhất quan sát trong một lần chạy DFS và độ sâu tối đa về mặt lý thuyết. Một lượt DFS chịu ảnh hưởng bởi thứ tự duyệt và cách đánh dấu trạng thái đã thăm nên không thể đại diện cho mọi đường đi có thể tồn tại. Việc sử dụng cận trên m_upper cùng giá trị thực nghiệm m_DFS giúp báo cáo phản ánh đúng phạm vi của phép đo.

Từ bài thực hành có thể kết luận rằng mô hình hóa bài toán là bước nền tảng trước khi triển khai thuật toán tìm kiếm. Việc xác định thống nhất trạng thái, hành động, quy tắc chuyển trạng thái và chi phí giúp các thuật toán như BFS, DFS và A* xử lý cùng một môi trường, đồng thời tạo cơ sở để so sánh tính đầy đủ, tính tối ưu và hiệu suất trong các chương tiếp theo.

CHƯƠNG 3: Tìm kiếm không thông tin: BFS và DFS

3.1. Cơ sở lý thuyết

Tìm kiếm không thông tin (Uninformed Search) là nhóm phương pháp tìm kiếm không sử dụng hàm ước lượng khoảng cách đến đích để quyết định trạng thái cần mở rộng. Thuật toán lựa chọn nút dựa trên cách tổ chức danh sách chờ và thứ tự sinh các trạng thái kế tiếp. Trong Lab 02, Breadth-First Search (BFS) và Depth-First Search (DFS) được khảo sát trên cùng mô hình mê cung, trong đó tác tử di chuyển theo bốn hướng và mỗi hành động hợp lệ có chi phí bằng 1.

Hai thuật toán cùng xuất phát từ trạng thái S, tạo các nút con tương ứng với hành động hợp lệ và dừng khi tìm được trạng thái G hoặc khi điều kiện dừng được kích hoạt. Khác biệt cơ bản nằm ở nguyên tắc lựa chọn nút kế tiếp: BFS ưu tiên độ sâu nhỏ, còn DFS ưu tiên tiếp tục khám phá một nhánh trước khi quay lui.

3.1.1. Thuật toán Breadth-First Search (BFS)

BFS là thuật toán tìm kiếm theo chiều rộng, mở rộng các nút lần lượt theo từng mức độ sâu của cây tìm kiếm. Thuật toán bắt đầu từ nút gốc tại S, tiếp tục xét các trạng thái cách S một bước, sau đó hai bước và lần lượt đến các mức sâu hơn. Để duy trì thứ tự này, BFS sử dụng hàng đợi (Queue) theo nguyên tắc First-In, First-Out (FIFO): nút được thêm vào trước sẽ được lấy ra xử lý trước.

Trong quá trình thực hiện, một nút được lấy từ đầu hàng đợi để kiểm tra điều kiện đích. Nếu chưa đến G, chương trình xác định các hành động hợp lệ và bổ sung các trạng thái kế tiếp chưa được phát hiện vào cuối hàng đợi. Tập reached được cập nhật khi một trạng thái được phát hiện nhằm hạn chế việc thêm cùng một vị trí nhiều lần.

Vì các nút được xét theo độ sâu không giảm, BFS bảo đảm tìm được lời giải có số bước ít nhất trong không gian trạng thái hữu hạn với hệ số phân nhánh hữu hạn. Khi mọi hành động có chi phí bằng nhau như trong bài thực hành, lời giải ít bước nhất cũng là lời giải có chi phí tối ưu. Đổi lại, BFS có thể phải duy trì nhiều nút ở những mức độ sâu gần nhau trong hàng đợi và tập trạng thái đã phát hiện, khiến nhu cầu bộ nhớ tăng khi số trạng thái cần khảo sát lớn.

3.1.2. Thuật toán Depth-First Search (DFS)

DFS là thuật toán tìm kiếm theo chiều sâu, ưu tiên mở rộng nút ở cuối nhánh đang khảo sát. Thuật toán sử dụng ngăn xếp (Stack) theo nguyên tắc Last-In, First-Out (LIFO): nút được thêm sau sẽ được lấy ra trước. Khi gặp ngõ cụt hoặc không còn trạng thái kế tiếp hợp lệ trên nhánh hiện tại, DFS quay lui để tiếp tục khảo sát những lựa chọn khác.

Không giống BFS, DFS có thể đi sâu vào một nhánh dài trước khi xét các nhánh gần trạng thái bắt đầu hơn. Vì vậy, đường đi đầu tiên được tìm thấy không nhất thiết có số bước nhỏ nhất. Kết quả còn chịu ảnh hưởng mạnh từ thứ tự sinh nút con, bởi thứ tự lấy nút khỏi stack là thứ tự ngược với thứ tự đưa các nút vào.

Trong phần triển khai DFS được sử dụng cho notebook, cơ chế kiểm tra chu trình dựa trên các trạng thái xuất hiện trên đường đi hiện tại từ nút gốc đến nút đang xét. Cách kiểm tra này ngăn việc quay lại cùng một trạng thái trong một nhánh nhưng vẫn cho phép một vị trí được khảo sát lại ở nhánh khác. Nếu thiếu kiểm tra chu trình hoặc thuật toán có thể liên tục phát triển nhánh sâu, DFS có nguy cơ lặp lại thao tác hoặc không tìm được lời giải trong giới hạn tài nguyên đã đặt.

3.2. Triển khai thuật toán tìm kiếm

Phần thực hành sử dụng hai notebook 02_Maze_Example.ipynb và 03_Maze_Example_Animation.ipynb. Notebook thứ nhất thực hiện và so sánh BFS, DFS; notebook thứ hai trực quan hóa diễn biến mở rộng trạng thái bằng animation. Các kết quả thống kê được lấy từ phiên bản cài đặt hỗ trợ tree_search_solution.py đi kèm bộ thực hành, không phải mã nguồn gốc chưa được công bố của tác giả notebook.

3.2.1. Cấu trúc Queue và Stack

Với BFS, frontier được tổ chức bằng cấu trúc deque của Python và mỗi lần thuật toán thực hiện lấy nút bằng thao tác popleft(). Các nút con hợp lệ được bổ sung vào cuối hàng đợi bằng append(). Nhờ đó, trạng thái xuất hiện trong frontier sớm hơn được ưu tiên mở rộng trước, giúp thuật toán lần lượt duyệt các mức độ sâu.

Đối với DFS, frontier được tổ chức như một stack. Nút mới được đưa vào bằng append() và nút cần khảo sát tiếp theo được lấy bằng pop(). Trong notebook, thứ tự thêm trạng thái con được thiết lập bằng set_order("NESW"), tương ứng Bắc, Đông, Nam, Tây. Do stack hoạt động theo LIFO, hướng Tây có thể được lấy ra trước các hướng còn lại nếu tất cả các hướng đều hợp lệ và được đưa vào stack theo thứ tự này.

Ví dụ trên minh họa rằng thứ tự sinh nút con không phải thứ tự thực tế các nút được mở rộng trong DFS. Khi thay đổi thứ tự hướng hoặc bật chế độ sinh hướng ngẫu nhiên, đường đi tìm được và số nút mở rộng có thể thay đổi ngay cả khi bản đồ đầu vào không thay đổi.

3.2.2. Quản lý frontier và trạng thái mở rộng

Frontier là tập các nút đã được phát hiện nhưng chưa được mở rộng. Mỗi lần lấy một nút hợp lệ khỏi frontier, chương trình kiểm tra trạng thái đích; nếu chưa đạt đích, thuật toán xét các hành động hợp lệ để sinh các nút tiếp theo. Các nút được liên kết thông qua thuộc tính parent nhằm phục vụ truy hồi đường đi sau khi tìm thấy G.

Ở BFS, reached lưu các trạng thái được phát hiện và được sử dụng để loại bỏ trạng thái lặp trước khi bổ sung vào queue. Ở DFS, việc loại bỏ chu trình trong lần chạy thực nghiệm được thực hiện bằng cách kiểm tra tập các trạng thái trên đường đi hiện tại. Các vị trí đã mở rộng có thể được ghi lại phục vụ thống kê và hiển thị, nhưng không tương đương với một tập visited toàn cục được dùng để cắt nhánh DFS.

Các chỉ số được thu thập gồm path cost (tổng chi phí đường đi), n_expanded (số lần mở rộng nút), max_frontier (số nút lớn nhất có mặt đồng thời trong frontier) và

max_depth (độ sâu lớn nhất của những nút được mở rộng, theo quy ước thống kê của mã thực nghiệm). Nút đích được kiểm tra trước bước sinh trạng thái con nên không được cộng vào n_expanded. Chương trình còn lưu trace để quan sát thứ tự mở rộng và biến động của frontier.

Để bảo đảm phép so sánh có thể tái lập, phần khảo sát chính sử dụng cùng dữ liệu mê cung, chi phí bước bằng 1 và thứ tự thêm nút NESW. Đối với DFS ngẫu nhiên, mỗi lần chạy được gắn một seed cố định. Các lượt tìm kiếm đều đặt giới hạn max_tries nhằm tránh chạy quá lâu trong trường hợp thuật toán liên tục mở rộng nhiều nhánh.

3.3. Kết quả thực nghiệm

3.3.1. Kết quả thực nghiệm BFS

Thực nghiệm BFS được tiến hành trên small_maze, với điểm bắt đầu S = (3, 11) và điểm đích G = (8, 1). Thuật toán tìm được lời giải có độ dài 19 bước và chi phí đường đi bằng 19. Trong quá trình tìm kiếm, chương trình mở rộng 92 nút, còn kích thước lớn nhất của frontier là 9. Độ sâu lớn nhất được ghi nhận trong lượt chạy này là 19.

Chuỗi hành động tìm được là E → E → S → S → W → S → S → W → W → S → W → W → W → W → W → W → W → W → W. Chuỗi này bắt đầu tại S, di chuyển qua những ô hợp lệ và kết thúc tại G. Vì BFS xét các đường đi theo độ sâu tăng dần, kết quả 19 bước là lời giải ngắn nhất trong thiết lập bốn hướng có chi phí mỗi bước bằng 1.

Khi theo dõi trace của BFS, số phần tử frontier thay đổi sau mỗi lần mở rộng do một trạng thái có thể sinh ra nhiều nút con hợp lệ hoặc không sinh thêm trạng thái mới. Dù kích thước frontier không tăng đều, thuật toán vẫn bảo toàn nguyên tắc FIFO, tức các nút ở độ sâu nhỏ hơn được ưu tiên trước các nút ở độ sâu lớn hơn. Trong notebook animation, quá trình này được biểu diễn bằng các khung hình thể hiện vùng đã khảo sát, frontier và đường đi cuối cùng.

3.3.2. Kết quả thực nghiệm DFS qua nhiều lần chạy

Với cùng mê cung small và thứ tự thêm nút NESW, DFS tìm được đường đi dài 29 bước, mở rộng 39 nút và có kích thước frontier lớn nhất bằng 5. So với BFS, số nút mở rộng và kích thước frontier trong lượt chạy này thấp hơn, nhưng đường đi dài hơn 10 bước. Đây là bằng chứng thực nghiệm cho thấy DFS không bảo đảm tìm được lời giải tối ưu dù có thể tìm thấy đích sau khi khảo sát ít nút hơn.

Để xác định ảnh hưởng của thứ tự sinh trạng thái, notebook tiếp tục thực hiện tám lần DFS với thứ tự hướng được xáo trộn và các giá trị seed cố định. Kết quả được trình bày trong Bảng 3.1.

Bảng 3.1. Kết quả tám lần chạy DFS ngẫu nhiên trên small_maze

Lần chạy	Seed	Path cost	Nút mở rộng	Max frontier
1	20261010	49	73	6
2	20261011	49	65	6
3	20261012	49	65	6
4	20261013	49	65	6
5	20261014	37	37	7
6	20261015	37	79	6

7	20261016	19	45	6
8	20261017	19	35	5

Cả tám lượt chạy đều tìm được đích, nhưng độ dài đường đi biến động từ 19 đến 49 bước. Chỉ hai lượt chạy cuối có đường đi 19 bước, bằng lời giải tối ưu của BFS. Các lượt còn lại cho kết quả 37 hoặc 49 bước. Số nút mở rộng cũng biến động trong khoảng từ 35 đến 79, chứng tỏ việc thay đổi thứ tự lựa chọn nhánh ảnh hưởng đồng thời đến chất lượng lời giải và khối lượng tìm kiếm của DFS.

Kết quả trực quan hóa trong 03_Maze_Example_Animation.ipynb cũng ghi nhận lượt chạy DFS ngẫu nhiên với seed 20261010 có đường đi 49 bước, mở rộng 73 nút và max_frontier bằng 6; trong cùng điều kiện dữ liệu, BFS vẫn tìm được lời giải 19 bước. Do animation chỉ lưu các khung hình được chọn mẫu, số frame lưu không phản ánh trực tiếp số nút đã mở rộng.

3.4. So sánh BFS và DFS

Ngoài thí nghiệm trên small, notebook thực hiện BFS và DFS với thứ tự thêm nút NESW trên các mê cung medium và L. Các giá trị trong Bảng 3.2 được trích từ kết quả chạy và bảng dữ liệu ket_qua_bai3.csv.

Bảng 3.2. So sánh kết quả BFS và DFS trên ba mê cung

Mê cung	Thuật toán	Path cost	Nút mở rộng	Max frontier	Max depth
small	BFS	19	92	9	19

small	DFS	29	39	5	28
medium	BFS	68	269	9	68
medium	DFS	152	153	8	151
L	BFS	16	151	17	16
L	DFS	122	132	109	121

Về chất lượng lời giải, BFS tìm được đường đi ngắn nhất trên cả ba mê cung nhờ nguyên tắc xét nút theo từng mức độ sâu. DFS có đường đi dài hơn BFS ở tất cả các trường hợp trong bảng, đặc biệt trên mê cung L: đường đi DFS có 122 bước trong khi BFS chỉ cần 16 bước. Như vậy, DFS có thể tìm được đích nhưng không bảo đảm tối ưu về số bước hay chi phí.

Về số nút mở rộng, DFS ít hơn BFS trong ba trường hợp được khảo sát. Chẳng hạn trên medium, DFS mở rộng 153 nút so với 269 nút của BFS. Tuy nhiên, số nút mở rộng không phải là thời gian thực thi thực tế; thời gian còn chịu ảnh hưởng từ thao tác quản lý trạng thái, kiểm tra chu trình và chi phí lưu vết để trực quan hóa. Bảng thực nghiệm được sử dụng ở đây không cung cấp số đo thời gian nhất quán cho từng trường hợp, vì vậy không thể kết luận DFS luôn chạy nhanh hơn BFS.

Về bộ nhớ, BFS phải lưu các trạng thái chưa xét trong queue và quản lý tập reached, nên ở các bài toán có mức phân nhánh lớn có thể cần nhiều bộ nhớ. DFS theo cơ chế tìm kiếm trên cây thường chỉ cần lưu thông tin phục vụ các nhánh đang chờ và đường đi hiện tại, nhưng mức sử dụng bộ nhớ thực tế phụ thuộc cách cài đặt. Trên small và medium, max_frontier của DFS thấp hơn BFS; trái lại, trên mê cung L, DFS đạt max_frontier bằng 109, lớn hơn mức 17 của BFS. Do đó, không

thể suy ra DFS luôn sử dụng ít bộ nhớ hơn chỉ từ tên thuật toán hoặc chỉ số frontier của một lần chạy.

Về tính đầy đủ, BFS là phương pháp đầy đủ khi hệ số phân nhánh hữu hạn và có lời giải ở độ sâu hữu hạn; trong thiết lập chi phí bước bằng nhau, BFS còn bảo đảm tìm được lời giải tối ưu. DFS có thể gặp vấn đề với các nhánh rất sâu hoặc chu trình nếu không kiểm tra lặp. Trong mê cung hữu hạn, nếu DFS ngăn lặp trạng thái trên đường đi hiện tại và được phép duyệt hết các nhánh hữu hạn, thuật toán vẫn có thể tìm được lời giải; tuy nhiên, giới hạn max_tries có thể khiến việc tìm kiếm dừng trước khi đến đích.

Bảng 3.3. So sánh đặc điểm của hai thuật toán

Tiêu chí	**BFS**	**DFS**
Cấu trúc frontier	Queue, FIFO	Stack, LIFO
Quy tắc mở rộng	Theo độ sâu tăng dần	Ưu tiên đi sâu rồi quay lui
Tối ưu khi chi phí bước bằng nhau	Có	Không bảo đảm
Tính đầy đủ	Có trong điều kiện tìm kiếm hữu hạn phù hợp	Phụ thuộc xử lý chu trình, độ sâu và giới hạn chạy
Nhu cầu bộ nhớ	Có thể lớn do lưu nhiều nút và reached	Thường thuận lợi hơn với cách cài đặt tìm kiếm theo nhánh; không phải mọi trường hợp

Ảnh hưởng của thứ tự sinh nút	Có thể thay đổi đường đi đồng tối ưu và số nút mở rộng	Có thể thay đổi mạnh độ dài đường đi và số nút mở rộng

3.5. Nhận xét

Kết quả thực nghiệm cho thấy khác biệt giữa BFS và DFS chủ yếu xuất phát từ nguyên tắc tổ chức frontier. BFS sử dụng FIFO để duy trì thứ tự mở rộng theo độ sâu, nhờ đó bảo đảm tìm được đường đi ngắn nhất khi mọi bước có chi phí bằng nhau. DFS sử dụng LIFO nên ưu tiên trạng thái được bổ sung sau cùng và có thể đi sâu vào một nhánh chưa tối ưu trước khi xem xét những lựa chọn khác.

Trên small_maze, BFS tìm được lời giải 19 bước với 92 nút mở rộng, trong khi DFS với thứ tự NESW tìm được đường đi 29 bước và mở rộng 39 nút. Qua tám lần chạy DFS với thứ tự hướng ngẫu nhiên, kết quả dao động từ 19 đến 49 bước. Sự biến động này cho thấy thứ tự sinh trạng thái là yếu tố có ảnh hưởng lớn đến DFS; một lần chạy có thể tình cờ tìm được đường đi tối ưu, nhưng điều đó không đồng nghĩa thuật toán có bảo đảm tối ưu.

Ngoài ra, việc quản lý frontier và kiểm tra chu trình tác động trực tiếp đến kết quả thống kê. Mặc dù DFS thường được giới thiệu là có ưu thế về bộ nhớ khi chỉ lưu những nhánh cần khảo sát, thí nghiệm trên mê cung L ghi nhận max_frontier của DFS lớn hơn BFS. Do vậy, khi đánh giá bộ nhớ cần nêu rõ phương pháp cài đặt và định nghĩa chỉ số đo, tránh đồng nhất kích thước frontier với tổng bộ nhớ chương trình tiêu thụ.

Từ bài thực hành có thể kết luận rằng BFS phù hợp khi cần lời giải ngắn nhất và có thể chấp nhận chi phí lưu trữ trạng thái, còn DFS hữu ích khi muốn ưu tiên khảo sát sâu và có cơ chế xử lý chu trình, giới hạn tài nguyên phù hợp. Những kết quả này là cơ sở để nghiên cứu tiếp các thuật toán Depth-Limited DFS, Iterative Deepening Search và những phương pháp tìm kiếm sử dụng heuristic trong các chương sau.

CHƯƠNG 4: Depth-Limited DFS và Iterative Deepening Search

4.1. Thuật toán Depth-Limited DFS (DLS)

Depth-Limited Depth-First Search (DLS) là biến thể của tìm kiếm theo chiều sâu (DFS), trong đó việc mở rộng cây tìm kiếm bị giới hạn tại một độ sâu định trước. Phương pháp này được khảo sát trong notebook 02_Maze_Example.ipynb nhằm làm rõ tác động của giới hạn độ sâu đến khả năng tìm được đích, số nút mở rộng và chất lượng đường đi. Trên mô hình mê cung của Lab 02, trạng thái là tọa độ ô, các hành động di chuyển theo bốn hướng và mỗi bước hợp lệ có chi phí bằng 1.

4.1.1. Nguyên lý giới hạn độ sâu

Gọi L là giới hạn độ sâu và depth(n) là độ sâu của nút n tính từ nút gốc. DLS duyệt các nút theo nguyên tắc LIFO của DFS nhưng chỉ sinh nút con khi depth(n) < L. Nút ở đúng độ sâu L vẫn được kiểm tra xem có phải trạng thái đích hay không; nếu chưa phải đích thì nút không được mở rộng tiếp. Quy tắc này ngăn thuật toán tiếp tục đi vào các nhánh sâu hơn giới hạn cho phép.

Một lượt DLS có thể trả về ba loại kết quả cơ bản. FOUND biểu thị đã tìm được đích; CUTOFF biểu thị tìm kiếm chưa đạt đích và có ít nhất một nhánh bị cắt bởi giới hạn độ sâu; FAILURE biểu thị không tìm được đích sau khi đã khảo sát hết những đường đi liên quan mà không còn nhánh nào bị cắt ở giới hạn. CUTOFF không đồng nghĩa với việc mê cung không có lời giải, bởi tăng L có thể giúp tìm thấy đích ở một độ sâu lớn hơn.

Trong phần cài đặt bổ sung của notebook, mỗi nút lưu vị trí, nút cha, hành động tạo ra nút và độ sâu hiện tại. Frontier được tổ chức bằng stack. Việc truy hồi đường đi được thực hiện bằng các liên kết nút cha. Chỉ số n_expanded đếm số nút thực sự sinh trạng thái kế tiếp, không tính nút đích hoặc nút bị chặn ở độ sâu giới hạn.

Ngược lại, max_depth có thể ghi nhận một nút đã được lấy khỏi stack ở đúng độ sâu L.

4.1.2. Ảnh hưởng của giá trị giới hạn *L*

Chọn giới hạn L có tác động trực tiếp đến phạm vi tìm kiếm. Gọi d là độ sâu lời giải tối ưu. Nếu L < d, DLS không thể đến đích vì mọi đường đi hợp lệ đều dài hơn mức được phép khảo sát. Khi L = d, thuật toán có thể tìm được lời giải có số bước tối ưu nếu cơ chế duyệt và kiểm tra chu trình không loại bỏ nhầm đường đi hợp lệ. Nếu L > d, DLS có thể tìm được đích qua một nhánh dài hơn d trước khi xét đến đường đi ngắn nhất, do bản chất ưu tiên đi sâu của DFS.

Trên small_maze, BFS đã xác định độ sâu tối ưu d = 19. Vì vậy, thí nghiệm lựa chọn các giới hạn L = 0, 5, 10, 18, 19, 20 và 30 nhằm quan sát rõ ba trường hợp: giới hạn chưa đủ sâu, giới hạn vừa đủ và giới hạn lớn hơn độ sâu tối ưu. Kết quả tại L = 30 đặc biệt có ý nghĩa vì DLS tìm được đường đi 29 bước thay vì 19 bước, cho thấy tăng giới hạn không bảo đảm cải thiện chất lượng lời giải.

4.2. Thuật toán Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) kết hợp nhiều lượt DLS với các giá trị giới hạn tăng dần. Thay vì chọn trước một giá trị L duy nhất, IDS thực hiện các lần tìm kiếm từ độ sâu nhỏ nhất và chỉ tăng giới hạn khi lần tìm kiếm trước bị cắt mà chưa tìm được đích. Cách tiếp cận này phù hợp khi chưa biết độ sâu lời giải tối ưu của bài toán.

4.2.1. Nguyên lý tăng dần giới hạn độ sâu

Trong bài thực hành, IDS bắt đầu với L = 0, sau đó lần lượt tăng thành 1, 2, 3, … cho đến khi DLS trả về FOUND hoặc xuất hiện điều kiện dừng khác. Mỗi lần tăng giới hạn tạo một lượt DLS độc lập với stack mới. Vì vậy, các trạng thái ở tầng nông có thể được khảo sát lại qua nhiều lượt tìm kiếm.

Khi lời giải nông nhất nằm ở độ sâu d và các lượt DLS hoạt động đúng, IDS tìm đích lần đầu tại L = d. Nếu bắt đầu từ L = 0, thuật toán thực hiện d + 1 lượt DLS

nhưng chỉ có d lần tăng giới hạn. Hai đại lượng này cần được phân biệt khi ghi kết quả thực nghiệm. Trên small_maze, việc xét từ L = 0 đến L = 19 tương ứng với 20 lượt DLS và 19 lần tăng giới hạn.

Notebook ghi lại lịch sử từng lượt chạy, gồm mức L, trạng thái FOUND hoặc CUTOFF, số nút mở rộng trong lượt đó, số nút mở rộng cộng dồn và kích thước frontier lớn nhất. Tổng số nút mở rộng của IDS được tính bằng tổng số nút mở rộng ở tất cả lượt DLS, không chỉ lấy kết quả của lượt cuối cùng. Vì mỗi lượt có thể phải khảo sát lại các tầng nông, tổng công việc của IDS thường lớn hơn một lượt tìm kiếm với giới hạn đã biết sẵn.

4.2.2. Sự kết hợp các đặc tính của BFS và DFS

IDS kế thừa đặc tính tìm nghiệm theo độ sâu tăng dần tương tự BFS: thuật toán chưa chấp nhận lời giải ở độ sâu lớn khi vẫn chưa thử hết những giới hạn nhỏ hơn. Nhờ đó, khi chi phí các bước bằng nhau, IDS có thể tìm được lời giải có số bước ít nhất, với điều kiện DLS thành phần hoàn chỉnh trong từng ngưỡng và cơ chế kiểm tra chu trình phù hợp.

Về bộ nhớ, mỗi lượt IDS sử dụng stack và quan hệ nút cha như DFS; stack của lượt trước không cần được duy trì khi bắt đầu lượt sau. Đối với mô hình cây tìm kiếm với hệ số phân nhánh tối đa b, cách tổ chức DFS giới hạn độ sâu thường được mô tả có nhu cầu bộ nhớ theo frontier cỡ O(bL), trong khi IDS đến độ sâu nghiệm d có mức cỡ O(bd). Cách đánh giá này là mô hình lý thuyết của tìm kiếm trên cây, không phải số byte bộ nhớ thực tế của chương trình Python.

Ưu điểm bộ nhớ đi kèm chi phí tính toán do mở rộng lại những tầng đã khảo sát. Trong cài đặt thực tế, bộ nhớ còn phụ thuộc vào các đối tượng nút được giữ qua liên kết parent, dữ liệu thống kê và cấu trúc kiểm tra chu trình. Vì thế, chỉ số max_frontier phản ánh số nút đang chờ lớn nhất trong một lượt tìm kiếm chứ không đại diện đầy đủ cho toàn bộ bộ nhớ chương trình sử dụng.

4.3. Kết quả thực nghiệm DLS và IDS

Các thí nghiệm chính được thực hiện trên small_maze.txt, với trạng thái bắt đầu S = (3, 11), trạng thái đích G = (8, 1), thứ tự thêm trạng thái con NESW (Bắc, Đông, Nam, Tây) và cơ chế kiểm tra chu trình theo đường đi hiện tại. Do stack hoạt động theo LIFO, thứ tự lấy các nút khỏi frontier đảo ngược thứ tự bổ sung nếu những hành động tương ứng đều hợp lệ. Kết quả được lấy từ notebook và bảng ket_qua_bai4_DLS_IDS.csv, trong đó BFS được sử dụng làm mốc tham chiếu về độ sâu lời giải tối ưu.

4.3.1. Kết quả DLS với các giới hạn khác nhau

Bảng 4.1 thể hiện ảnh hưởng của giới hạn độ sâu đến kết quả DLS trên small_maze.

Bảng 4.1. Kết quả DLS khi thay đổi giới hạn độ sâu trên small_maze

Giới hạn L	Kết quả	Path cost	Nút mở rộng	Max frontier	Max depth
0	CUTOFF	—	0	1	0
5	CUTOFF	—	15	4	5
10	CUTOFF	—	44	5	10

18	CUTOFF	—	114	6	18
19	FOUND	19	72	6	19
20	FOUND	19	79	6	20
30	FOUND	29	39	5	29

Khi L = 0, 5, 10 hoặc 18, DLS chưa thể tìm được đích và kết thúc với trạng thái CUTOFF. Đến L = 19, thuật toán tìm được đường đi 19 bước, bằng kết quả tối ưu do BFS xác định. Khi tăng L lên 20, thuật toán vẫn tìm được đường đi 19 bước nhưng số nút mở rộng tăng từ 72 lên 79 vì phạm vi duyệt cho phép đi sâu hơn trước khi gặp đích.

Đáng chú ý, tại L = 30, DLS chỉ mở rộng 39 nút nhưng trả về đường đi dài 29 bước. Như vậy, số nút mở rộng ít hơn không đồng nghĩa với lời giải tốt hơn. Số nút mở rộng cũng không tăng đơn điệu theo giới hạn L, vì việc cho phép đi sâu có thể khiến DFS gặp đích trên một nhánh khác và dừng sớm. Kết quả trên minh họa rõ sự đánh đổi giữa phạm vi tìm kiếm và tính tối ưu của DLS.

4.3.2. Số lần tăng giới hạn độ sâu của IDS

IDS trên small_maze thực hiện lần lượt các ngưỡng L = 0 đến L = 19 và dừng khi tìm được đích. Bảng 4.2 ghi lại số nút mở rộng của từng lượt, số mở rộng cộng dồn và kích thước frontier lớn nhất của lượt đó.

Bảng 4.2. Lịch sử tăng giới hạn của IDS trên small_maze

L	Kết quả	Nút mở rộng/lượt	Cộng dồn	Max frontier
0	CUTOFF	0	0	1
1	CUTOFF	1	1	3
2	CUTOFF	4	5	3
3	CUTOFF	7	12	3
4	CUTOFF	11	23	3
5	CUTOFF	15	38	4
6	CUTOFF	19	57	5
7	CUTOFF	24	81	5
8	CUTOFF	29	110	5
9	CUTOFF	36	146	5

10	CUTOFF	44	190	5
11	CUTOFF	53	243	5
12	CUTOFF	61	304	5
13	CUTOFF	69	373	6
14	CUTOFF	78	451	6
15	CUTOFF	87	538	6
16	CUTOFF	95	633	6
17	CUTOFF	104	737	6
18	CUTOFF	114	851	6
19	FOUND	72	923	6

Sau 20 lượt DLS, IDS tìm được lời giải có chi phí bằng 19 và số lần tăng giới hạn bằng 19. Tổng số nút mở rộng trong toàn bộ quá trình là 923, trong khi max_frontier lớn nhất của một lượt chỉ bằng 6. Kết quả xác nhận ưu điểm về giới hạn số nút đang chờ nhưng cũng cho thấy sự lặp lại công việc qua các lần tăng độ

sâu. Số nút mở rộng ở lượt cuối là 72, không phải 923; giá trị 923 là tổng mở rộng của toàn bộ IDS.

Để kiểm tra kết luận trên một bài toán khác, notebook thực hiện khảo sát bổ sung với medium_maze. BFS cho đường đi ngắn nhất 68 bước, còn IDS cũng tìm được đường 68 bước sau khi tăng giới hạn 68 lần, tương ứng 69 lượt DLS. Tổng số nút mở rộng của IDS trên medium_maze là 15.665 và max_frontier bằng 8. Bảng 4.3 đối chiếu kết quả hai mê cung.

Bảng 4.3. Tổng hợp kết quả IDS trên small_maze và medium_maze

Mê cung	Path cost	Lượt DLS	Lần tăng L	Tổng nút mở rộng	Max frontier
small	19	20	19	923	6
medium	68	69	68	15.665	8

Những số liệu trên không phải thời gian thực thi tính bằng mili giây. Số lượt tìm kiếm và số nút mở rộng cho phép đánh giá khối lượng công việc, còn so sánh tốc độ thực tế cần thực hiện phép đo thời gian trong cùng môi trường và cấu hình chạy.

4.4. Ảnh hưởng của cycle checking

Trong mê cung có chu trình, cùng một trạng thái có thể xuất hiện trên nhiều đường đi khác nhau. Để tránh tìm kiếm lặp vô ích, DFS thường cần cơ chế kiểm tra chu trình. Đối với DLS và IDS, cách kiểm tra đặc biệt quan trọng vì một trạng thái có thể được tiếp cận tại nhiều độ sâu, trong khi việc đạt đích còn phụ thuộc vào số bước còn được phép đi tiếp trước giới hạn L.

Trong phiên bản DLS được bổ sung vào notebook, mỗi trạng thái kế tiếp được so sánh với những trạng thái trên chuỗi tổ tiên của nút hiện tại. Nếu trạng thái đã có

trên đường đi hiện tại, chương trình không sinh nút con đó. Cơ chế này ngăn một đường đi tự quay lại trạng thái cũ nhưng không loại bỏ vĩnh viễn một trạng thái chỉ vì nó từng xuất hiện ở nhánh khác. Vì vậy, thuật toán vẫn có cơ hội tiếp cận cùng một vị trí qua đường đi khác có độ sâu ngắn hơn.

Notebook đồng thời xây dựng một trường hợp đối chứng không phù hợp: DLS sử dụng tập visited toàn cục, đánh dấu trạng thái ngay khi sinh nút con và không xem xét độ sâu tiếp cận. Trong thí nghiệm với small_maze tại L = 19, DLS kiểm tra chu trình theo đường đi hiện tại tìm được đích với chi phí 19, nhưng phiên bản visited toàn cục đơn giản trả về NO_SOLUTION. Kết quả NO_SOLUTION này không chứng minh mê cung vô nghiệm; đây là hiện tượng bỏ sót đường đi do cách loại bỏ trạng thái không phù hợp với giới hạn độ sâu.

Nguyên nhân là một trạng thái có thể được phát hiện trước thông qua đường đi dài và bị cắt ở mức L, trong khi một đường đi ngắn hơn đến cùng trạng thái có thể còn đủ độ sâu để tiếp tục đến đích. Nếu tập visited toàn cục chặn mọi lần gặp lại trạng thái đó, thuật toán sẽ bỏ qua khả năng tìm kiếm hợp lệ. Điều này không có nghĩa mọi hình thức quản lý trạng thái toàn cục đều sai; một cơ chế ghi nhận độ sâu tốt nhất và cho phép cập nhật khi tìm được đường đi nông hơn có thể được thiết kế phù hợp hơn.

Với IDS, lỗi ở một lượt DLS có thể ảnh hưởng đến kết quả của cả quá trình tăng giới hạn. Do đó, việc tái sử dụng DLS trong IDS cần bảo đảm kiểm tra chu trình không loại bỏ các đường đi cần thiết. Bên cạnh khả năng tránh lặp, cycle checking cũng phát sinh chi phí quản lý tham chiếu và so sánh trạng thái. Đây là sự đánh đổi cần cân nhắc khi triển khai các thuật toán trên không gian tìm kiếm phức tạp.

4.5. Nhận xét

Qua thực nghiệm, DLS cho thấy giới hạn độ sâu là công cụ trực tiếp để kiểm soát phạm vi phát triển của DFS. Với small_maze, khi L nhỏ hơn độ sâu lời giải tối ưu 19, thuật toán chưa tìm được đích; tại L = 19, lời giải 19 bước được xác định; còn tại L = 30, thuật toán tìm được đường đi 29 bước. Kết quả này cho thấy việc tăng giới hạn không đồng nghĩa với cải thiện chất lượng lời giải, bởi DLS vẫn phụ thuộc vào thứ tự duyệt sâu của stack.

IDS khắc phục khó khăn phải lựa chọn trước một giới hạn phù hợp bằng cách thử lần lượt từ độ sâu 0. Trên cả small_maze và medium_maze, thuật toán tìm được đường đi có chi phí trùng với kết quả tối ưu của BFS khi chi phí mỗi bước bằng nhau. Đồng thời, chỉ số max_frontier ghi nhận ở các lượt IDS tương đối thấp so với tổng khối lượng công việc, phản ánh ưu thế của việc dùng lại cấu trúc DFS giới hạn độ sâu thay vì giữ tất cả các tầng tìm kiếm cùng lúc.

Tuy nhiên, ưu điểm về bộ nhớ đi kèm việc duyệt lại các tầng nông. IDS trên small_maze có tổng cộng 923 lượt mở rộng nút qua 20 lần chạy DLS, còn trên medium_maze số nút mở rộng cộng dồn đạt 15.665. Vì thế, IDS phù hợp khi cần lời giải có độ sâu tối ưu mà chưa biết trước độ sâu đích, nhưng phải chấp nhận chi phí tính toán lặp lại.

Bài thực hành cũng làm rõ vai trò quyết định của kiểm tra chu trình. Việc chỉ dựa vào một tập visited toàn cục đơn giản có thể bỏ sót lời giải trong DLS do không phân biệt độ sâu tiếp cận trạng thái. Kiểm tra chu trình theo đường đi hiện tại trong bản cài đặt thực nghiệm giúp duy trì các đường đi hợp lệ, dù vẫn có thể khảo sát lại cùng một vị trí trên các nhánh khác nhau.

Từ các kết quả trên, có thể kết luận rằng DLS phù hợp khi bài toán có giới hạn độ sâu đã biết hoặc cần khống chế tìm kiếm, trong khi IDS là lựa chọn hợp lý khi muốn tìm nghiệm theo độ sâu tăng dần và hạn chế lượng thông tin frontier phải lưu đồng thời. Hiệu quả thực tế của cả hai thuật toán phụ thuộc vào thứ tự sinh nút, cấu trúc mê cung và cơ chế cycle checking. Đây là cơ sở để nghiên cứu các chiến lược tìm kiếm có thông tin sử dụng heuristic ở các chương tiếp theo.

CHƯƠNG 5: Tìm kiếm có thông tin: GBFS, A* và Weighted A*

5.1. Cơ sở lý thuyết các thuật toán tìm kiếm có thông tin

Tìm kiếm có thông tin (Informed Search) sử dụng tri thức bổ sung về bài toán dưới dạng hàm heuristic để định hướng quá trình lựa chọn trạng thái cần mở rộng. Khác

với BFS chỉ xét thứ tự độ sâu, các thuật toán tìm kiếm có thông tin đánh giá mức độ triển vọng của nút dựa trên chi phí đã thực hiện, khoảng cách ước lượng đến đích hoặc sự kết hợp của hai thành phần này. Trong Lab 02, Greedy Best-First Search (GBFS), A* và Weighted A* được khảo sát trên bài toán tìm đường trong mê cung, với tập hành động bốn hướng và chi phí mỗi bước bằng 1.

Các thuật toán sử dụng chung mô hình trạng thái, bản đồ mê cung và điều kiện đích, nhưng có quy tắc ưu tiên khác nhau khi lấy nút từ frontier. Nhờ đó, quá trình thực nghiệm có thể đánh giá mức độ ảnh hưởng của heuristic đến số nút mở rộng, độ dài đường đi và kích thước frontier. BFS được sử dụng làm mốc đối chiếu vì bảo đảm đường đi ít bước nhất trong thiết lập chi phí đồng nhất.

5.1.1. Greedy Best-First Search (GBFS)

Greedy Best-First Search là phương pháp ưu tiên mở rộng nút có giá trị heuristic nhỏ nhất. Hàm đánh giá của GBFS được biểu diễn bằng f(n) = h(n), trong đó h(n) ước lượng chi phí còn lại từ trạng thái n đến trạng thái đích. Thuật toán hướng về những trạng thái được ước lượng gần đích mà không trực tiếp cân nhắc chi phí đã đi từ điểm bắt đầu.

Trong bài toán mê cung, khi sử dụng khoảng cách Manhattan, GBFS có xu hướng ưu tiên các ô làm giảm tổng chênh lệch hàng và cột so với đích. Cách lựa chọn này có thể giúp tiếp cận đích sau ít lần mở rộng, nhưng cũng dễ bị ảnh hưởng bởi các bức tường: một ô gần đích theo tọa độ chưa chắc nằm trên đường đi ngắn nhất vì có thể phải đi vòng qua vật cản.

Do chỉ xét h(n), GBFS không bảo đảm tính tối ưu. Việc chọn một đường đi có heuristic giảm nhanh có thể khiến thuật toán tìm được đích thông qua một nhánh dài trước khi khảo sát đường đi ngắn hơn. Kết quả trên small_maze ở mục 5.3 minh họa rõ hạn chế này.

5.1.2. Thuật toán A*

A* là thuật toán tìm kiếm có thông tin sử dụng đồng thời chi phí đã tích lũy và chi phí ước lượng đến đích. Mỗi nút được đánh giá theo biểu thức f(n) = g(n) + h(n), trong đó g(n) là tổng chi phí từ trạng thái đầu đến n, còn h(n) là ước lượng chi phí tối thiểu còn lại từ n đến đích. Thuật toán ưu tiên mở rộng nút có giá trị f(n) nhỏ nhất.

So với GBFS, A* không chỉ quan tâm trạng thái hiện tại có vẻ gần đích hay không mà còn xét chi phí đã bỏ ra để đến trạng thái đó. Cơ chế này giúp hạn chế việc theo đuổi các đường đi quá dài chỉ vì có khoảng cách ước lượng nhỏ. Trên mê cung bốn hướng với chi phí mỗi bước bằng 1, hai heuristic Manhattan và Euclidean đều không đánh giá vượt chi phí đường đi tối ưu còn lại. Chúng cũng thỏa điều kiện nhất quán đối với những bước di chuyển hợp lệ.

Với heuristic phù hợp và cách quản lý trạng thái cho phép xử lý đường đi tốt hơn khi cần thiết, A* bảo đảm tìm được lời giải tối ưu. Trong phiên bản thực nghiệm, các trạng thái được quản lý bằng chi phí g tốt nhất đã ghi nhận; nút cũ có chi phí cao hơn có thể được bỏ qua khi lấy khỏi hàng đợi ưu tiên. Điều này giúp thuật toán kiểm soát các đường đi đến cùng một vị trí.

5.1.3. Thuật toán Weighted A*

Weighted A* mở rộng nguyên lý của A* bằng cách nhân giá trị heuristic với trọng số w. Hàm đánh giá được biểu diễn bằng $f_w(n) = g(n) + w \times h(n)$. Khi $w = 1$, biểu thức trở về A* thông thường; khi $w > 1$, ảnh hưởng của ước lượng khoảng cách đến đích được tăng lên, khiến thuật toán có xu hướng ưu tiên các trạng thái có heuristic nhỏ.

Việc tăng w có thể giúp giảm khối lượng tìm kiếm trong một số trường hợp nhưng không bảo đảm tìm được đường đi tối ưu. Với w đủ lớn, cách lựa chọn nút của Weighted A* có xu hướng gần với GBFS vì thành phần h(n) trở nên nổi bật so với g(n). Ngược lại, trường hợp $0 < w < 1$ làm giảm vai trò của heuristic, khiến việc đánh giá thiên về chi phí đã đi hơn. Trường hợp này được đưa vào notebook như một thí nghiệm mở rộng để quan sát ảnh hưởng của trọng số.

Do kết quả còn phụ thuộc cấu trúc mê cung và thứ tự ưu tiên giữa các nút có cùng giá trị đánh giá, tăng w không đồng nghĩa với việc số nút mở rộng hay thời gian thực thi luôn giảm. Vì vậy, cần khảo sát nhiều giá trị w trên cùng một bộ dữ liệu thay vì suy luận chỉ từ công thức.

5.2. Hàm đánh giá và heuristic

5.2.1. Các thành phần $g(n), h(n), f(n)$

Trong bài toán tìm đường, g(n) là chi phí tích lũy của các hành động từ S đến nút n. Do mỗi lần di chuyển hợp lệ có chi phí bằng 1, g(n) bằng số bước đã thực hiện trên đường đi dẫn đến nút đó. Thành phần h(n) là ước lượng về chi phí còn lại từ n đến G, được tính bằng một heuristic xác định trước và không cần tìm toàn bộ đường đi đến đích.

Hàm f(n) kết hợp các thông tin này để quyết định thứ tự mở rộng của nút. GBFS dùng f(n) = h(n); A* dùng f(n) = g(n) + h(n); Weighted A* dùng f_w(n) = g(n) + w × h(n). Ba hàm đánh giá được triển khai trên cùng cấu trúc frontier ưu tiên nhưng dẫn đến những chiến lược tìm kiếm khác nhau.

Bảng 5.1. So sánh hàm đánh giá của các thuật toán

Thuật toán	Hàm đánh giá	Thành phần được ưu tiên
GBFS	f(n) = h(n)	Chi phí ước lượng còn lại
A*	f(n) = g(n) + h(n)	Chi phí đã đi và chi phí còn lại

Weighted A*	f_w(n) = g(n) + w × h(n)	Hai thành phần, điều chỉnh bằng w

Trong phiên bản mã nguồn hỗ trợ, các thuật toán có thông tin sử dụng hàng đợi ưu tiên để lấy nút có giá trị đánh giá nhỏ nhất. Chỉ số nodes expanded chỉ đếm những nút được lấy ra để sinh trạng thái kế tiếp, không tính nút đích ngay khi phát hiện qua kiểm tra ở đầu vòng lặp. Max frontier là số phần tử lớn nhất có mặt đồng thời trong hàng đợi chờ, không đại diện đầy đủ cho toàn bộ bộ nhớ chương trình.

5.2.2. Khoảng cách Manhattan và Euclidean

Heuristic Manhattan đo khoảng cách trên lưới bằng tổng độ chênh lệch tuyệt đối theo hàng và cột. Với trạng thái hiện tại n = (r_n, c_n) và trạng thái đích G = (r_g, c_g), công thức được xác định như sau:

$$h_M(n) = |r_n - r_g| + |c_n - c_g|$$

Heuristic Euclidean đo khoảng cách đường thẳng giữa hai vị trí trên mặt phẳng tọa độ và được xác định bằng:

$$h_E(n) = \sqrt{[(r_n - r_g)^2 + (c_n - c_g)^2]}$$

Đối với mô hình chỉ di chuyển lên, xuống, trái và phải, không cho phép di chuyển chéo, Manhattan phù hợp trực tiếp với số bước tối thiểu cần thực hiện khi bỏ qua tường. Trong khi đó, Euclidean tính khoảng cách đường thẳng nên thường cho giá trị nhỏ hơn hoặc bằng Manhattan. Với chi phí mỗi bước bằng 1, cả hai đều là cận dưới hợp lệ của chi phí tối ưu còn lại, nhưng Manhattan thường cung cấp ước lượng chặt hơn.

Ví dụ, nếu một trạng thái cách đích 3 ô theo hàng và 4 ô theo cột thì h_M(n) = 7, còn h_E(n) = 5. Khi chỉ được di chuyển bốn hướng, tác tử cần ít nhất 7 bước trong môi trường không có vật cản. Sự khác nhau giữa hai giá trị lý giải vì sao A* dùng Manhattan có thể tập trung việc mở rộng vào những trạng thái triển vọng hơn so với Euclidean trong các thí nghiệm của Lab 02.

Tuy nhiên, hiệu quả về số nút mở rộng còn phụ thuộc cách xử lý các nút có cùng giá trị f(n), cơ chế cập nhật chi phí và đặc điểm bản đồ. Do đó, đánh giá heuristic cần dựa đồng thời trên điều kiện lý thuyết và số liệu chạy thực tế.

5.3. Kết quả thực nghiệm

Các thí nghiệm được thực hiện trong notebook 02_Maze_Example.ipynb và 04_Maze_BFS_vs_A_Star.ipynb, sử dụng phiên bản mã nguồn hỗ trợ tree_search_solution.py cùng các file mê cung. Thiết lập chung là di chuyển theo bốn hướng, chi phí mỗi bước bằng 1, thứ tự sinh trạng thái cố định NESW và cùng quy ước đếm nút mở rộng. Bảng số liệu được đối chiếu với hai tệp kết quả ket_qua_bai5_BFS_GBFS_AStar.csv và ket_qua_bai5_weighted_AStar.csv.

5.3.1. So sánh GBFS, A* và BFS

Bảng 5.2 tổng hợp kết quả trên năm mê cung small, medium, L, open và large. BFS được dùng làm chuẩn độ dài đường đi tối ưu, GBFS sử dụng Manhattan và A* được chạy riêng với Manhattan và Euclidean. Path cost biểu diễn tổng chi phí đường đi, còn Nodes expanded là số lượt mở rộng nút theo quy ước đã nêu.

*Bảng 5.2. Kết quả so sánh BFS, GBFS và A** trên năm mê cung

Mê cung	Thuật toán	Path cost	Nodes expanded	Max frontier
small	BFS	19	92	9
small	GBFS – Manhattan	29	39	5

small	A* – Euclidean	19	55	7
small	A* – Manhattan	19	51	7
medium	BFS	68	269	9
medium	GBFS – Manhattan	152	155	8
medium	A* – Euclidean	68	224	8
medium	A* – Manhattan	68	217	8
L	BFS	16	151	17
L	GBFS – Manhattan	16	42	32
L	A* – Euclidean	16	62	27
L	A* – Manhattan	16	40	29

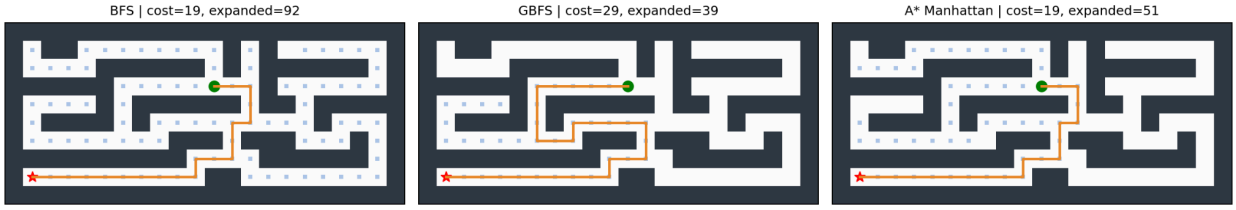
open	BFS	54	682	25
open	GBFS – Manhattan	54	772	64
open	A* – Euclidean	54	516	51
open	A* – Manhattan	54	211	185
large	BFS	210	621	8
large	GBFS – Manhattan	210	456	25
large	A* – Euclidean	210	554	10
large	A* – Manhattan	210	542	12

Trên small_maze, BFS tìm được đường đi ngắn nhất 19 bước sau 92 lần mở rộng nút. GBFS chỉ mở rộng 39 nút nhưng trả về đường đi 29 bước, dài hơn nghiệm tối ưu 10 bước. Ngược lại, A* dùng Manhattan và Euclidean đều giữ được đường đi 19 bước; số nút mở rộng lần lượt là 51 và 55. Kết quả này phản ánh lợi ích của việc kết hợp chi phí đã đi g(n) với ước lượng h(n), thay vì chỉ dựa vào độ gần đích theo tọa độ.

Trên medium_maze, sự chênh lệch về chất lượng lời giải còn rõ hơn. GBFS tìm đường dài 152 bước, trong khi BFS và cả hai biến thể A* đều tìm đường 68 bước.

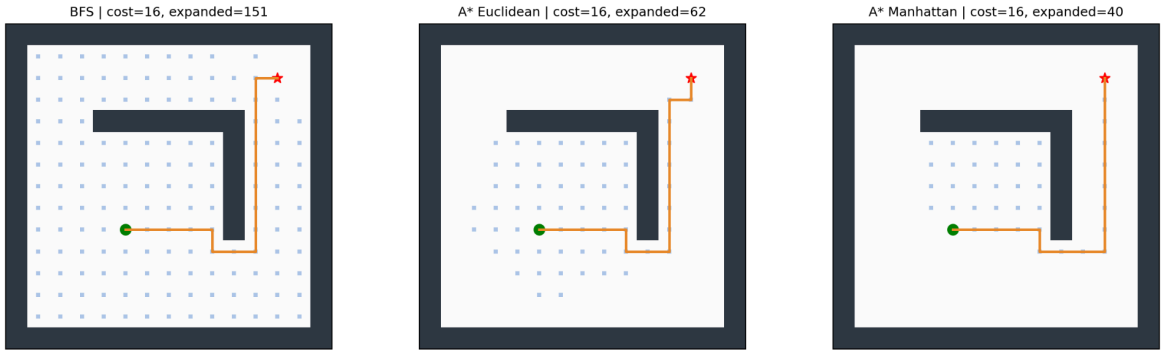
A* dùng Manhattan mở rộng 217 nút, ít hơn BFS là 52 nút. Các kết quả củng cố nhận xét rằng heuristic có thể giảm công sức tìm kiếm, nhưng cách lựa chọn chỉ theo h(n) không bảo đảm chất lượng đường đi.

Hình 5.1 trình bày đường đi và vùng trạng thái được khảo sát trên small_maze. Trong hình, đường màu cam thể hiện lời giải, ký hiệu đầu và cuối biểu diễn S và G; các chấm xanh nhạt biểu thị những vị trí được mở rộng. Đường đi GBFS vòng qua một phần mê cung nhiều hơn so với BFS và A*, tương ứng với chi phí 29 thay vì 19.



Hình 5.1. So sánh đường đi và vùng trạng thái được khảo sát bởi BFS, GBFS và A Manhattan trên small_maze.*

Trên L_maze, BFS và A* đều tìm đường 16 bước, nhưng BFS mở rộng 151 nút; A* – Euclidean mở rộng 62 nút và A* – Manhattan chỉ mở rộng 40 nút. GBFS trong trường hợp này cũng tìm được đường 16 bước với 42 nút mở rộng. Điều đó cho thấy GBFS có thể tình cờ tìm được lời giải tối ưu ở một mê cung cụ thể, nhưng kết quả này không làm thay đổi tính chất không bảo đảm tối ưu của thuật toán.



Hình 5.2. So sánh BFS, A Euclidean và A* Manhattan trên L_maze.*

Đối với open_maze, A* – Manhattan giảm số nút mở rộng từ 682 của BFS xuống còn 211, trong khi A* – Euclidean mở rộng 516 nút; cả ba cùng có path cost bằng 54. Tuy vậy, max_frontier của A* – Manhattan đạt 185, lớn hơn nhiều so với 25 của BFS. Vì thế, ít nút mở rộng hơn không đồng nghĩa frontier luôn nhỏ hơn. Kết quả GBFS trên open còn ghi nhận 772 lượt mở rộng, cao hơn tổng số ô hợp lệ của mê cung; đây là số lượt mở rộng nút chứ không phải số trạng thái phân biệt, do cách triển khai có thể mở rộng lại cùng một vị trí qua những đường đi khác nhau.

5.3.2. Khảo sát Weighted A* với các trọng số *w* khác nhau

Để khảo sát tác động của trọng số, Weighted A* được chạy trên cùng small_maze với heuristic Manhattan và các giá trị w = 0,5; 1; 1,5; 2; 3; 5; 10. Kết quả được trình bày trong Bảng 5.3. Tỷ lệ tăng chi phí lời giải được tính bằng chênh lệch giữa path cost của Weighted A* và chi phí tối ưu 19 bước, chia cho 19 rồi nhân 100%.

Bảng 5.3. Ảnh hưởng của trọng số w đến kết quả Weighted A trên small_maze*

Trọng số w	Path cost	Nodes expanded	Max frontier	Chi phí tăng so với tối ưu
0,5	19	67	8	0,00%
1	19	51	7	0,00%
1,5	19	57	6	0,00%
2	19	59	5	0,00%

3	19	60	5	0,00%
5	29	41	5	52,63%
10	29	39	5	52,63%

Khi w = 1, Weighted A* tương đương A* thông thường, tìm đường 19 bước và mở rộng 51 nút. Ở các giá trị w từ 1,5 đến 3, thuật toán vẫn giữ lời giải 19 bước, nhưng số nút mở rộng tăng lần lượt lên 57, 59 và 60. Như vậy, tăng trọng số trong khoảng này chưa cải thiện chỉ số mở rộng nút và không tạo ra lợi ích đơn điệu.

Khi tăng w lên 5 và 10, số nút mở rộng giảm còn 41 và 39, nhưng chi phí đường đi tăng lên 29 bước, tức cao hơn nghiệm tối ưu khoảng 52,63%. Kết quả cho thấy một trọng số lớn có thể làm thuật toán ưu tiên đi theo các ước lượng gần đích và tìm được lời giải sau ít lần mở rộng hơn, đồng thời đánh đổi bằng độ dài đường đi lớn hơn.

Mức max_frontier cũng biến đổi từ 8 tại w = 0,5 xuống 5 từ w = 2 trở lên trong thí nghiệm này. Tuy nhiên, đây chỉ là số nút đang chờ lớn nhất, không phải mức tiêu thụ bộ nhớ tính theo byte. Hơn nữa, kết quả với small_maze không bảo đảm được lặp lại tương tự ở mọi mê cung: trong bảng thực nghiệm tổng hợp, Weighted A* với w = 5 trên L_maze vẫn tìm được chi phí 16, trong khi trên medium_maze tìm đường 102 bước so với tối ưu 68.

5.4. Đánh giá sự đánh đổi giữa hiệu suất và chất lượng lời giải

Kết quả thực nghiệm cho thấy cần phân biệt ít nhất ba khía cạnh khi đánh giá thuật toán: chất lượng lời giải, số nút được mở rộng và nhu cầu lưu trữ các nút đang chờ. Một thuật toán mở rộng ít nút hơn có thể giảm một phần công việc tìm kiếm, nhưng

điều đó không tự động chứng minh thời gian chạy thực tế ngắn hơn hoặc đường đi tốt hơn.

Trên small_maze, GBFS mở rộng 39 nút so với 92 nút của BFS, nhưng đường đi dài 29 bước thay vì 19 bước. Tương tự, Weighted A* với w = 10 mở rộng 39 nút, ít hơn A* thông thường 12 nút, song đánh đổi bằng đường đi dài hơn 10 bước. Vì vậy, nếu mục tiêu của hệ thống là bảo đảm đường đi ngắn nhất, những kết quả này không thể được xem là cải thiện toàn diện chỉ vì giảm số nút mở rộng.

Ngược lại, A* kết hợp chi phí tích lũy và heuristic có khả năng giữ nghiệm tối ưu đồng thời cắt giảm vùng tìm kiếm. Trên L_maze, A* – Manhattan mở rộng 40 nút so với 151 nút của BFS và vẫn tìm đường 16 bước. Trên open_maze, A* – Manhattan mở rộng 211 nút so với 682 nút của BFS, với cùng đường đi 54 bước. Đây là minh chứng trực tiếp cho lợi ích của heuristic khi biểu diễn phù hợp với mô hình di chuyển.

Tuy nhiên, số nút mở rộng không phản ánh đầy đủ nhu cầu bộ nhớ. Trên L_maze, max_frontier của A* – Manhattan là 29, lớn hơn BFS là 17; trên open_maze, hai giá trị lần lượt là 185 và 25. Cách quản lý hàng đợi ưu tiên, số trạng thái chờ đồng thời và các bản ghi chi phí tốt nhất có thể làm bộ nhớ khác biệt ngay cả khi A* mở rộng ít nút hơn. Do đó, đánh giá bộ nhớ cần sử dụng max_frontier kết hợp với đặc điểm cài đặt, không suy ra trực tiếp từ nodes expanded.

Ngoài ra, dữ liệu CSV dùng trong báo cáo không lưu một bộ thời gian thực thi thống nhất cho tất cả các trường hợp. Vì thế, trong chương này, khái niệm hiệu suất được phân tích chủ yếu thông qua số nút mở rộng và max_frontier. Muốn so sánh tốc độ tính bằng thời gian, cần chạy đo lường nhiều lần trong cùng cấu hình phần cứng, cùng môi trường Python và cùng điều kiện thực nghiệm.

Từ các kết quả trên, có thể thấy lựa chọn thuật toán phụ thuộc yêu cầu của bài toán. GBFS hoặc Weighted A* với trọng số lớn có thể phù hợp trong tình huống chấp nhận lời giải chưa tối ưu để tìm kiếm theo định hướng heuristic mạnh hơn. A* phù hợp hơn khi cần lời giải tối ưu trong mô hình chi phí đã cho, còn BFS là phương pháp đối chiếu đáng tin cậy khi mọi chi phí bước bằng nhau nhưng không tận dụng thông tin về vị trí đích.

5.5. Nhận xét

Qua quá trình khảo sát, GBFS, A* và Weighted A* cho thấy những cách sử dụng heuristic khác nhau có ảnh hưởng trực tiếp đến quá trình tìm kiếm. GBFS tập trung vào ước lượng khoảng cách đến đích nên có thể tìm đường với ít nút mở rộng, nhưng không bảo đảm tối ưu khi vật cản khiến đường đi thực tế khác với khoảng cách ước lượng. Kết quả trên small_maze và medium_maze minh họa rõ hạn chế này.

A* bổ sung chi phí đã thực hiện g(n) vào hàm đánh giá, nhờ đó cân bằng giữa tiến độ hướng đến đích và chi phí tích lũy. Trong các mê cung khảo sát, A* dùng Manhattan và Euclidean đều tìm được path cost trùng BFS. Manhattan thường mở rộng ít nút hơn Euclidean do phù hợp hơn với chuyển động bốn hướng, nhưng hiệu quả về bộ nhớ không thể kết luận chỉ từ chỉ số mở rộng nút.

Weighted A* làm rõ sự đánh đổi giữa mức độ ưu tiên heuristic và chất lượng lời giải. Trên small_maze, tăng w từ 1 lên 2 không làm giảm số nút mở rộng, trong khi tăng lên 5 làm đường đi dài từ 19 lên 29 bước dù số lượt mở rộng giảm. Điều này cho thấy không tồn tại một lựa chọn trọng số luôn tốt nhất cho mọi mê cung hoặc mọi chỉ số đánh giá.

Nhìn chung, bài thực hành khẳng định vai trò của việc lựa chọn hàm đánh giá và heuristic phù hợp với cấu trúc bài toán. Trong mô hình mê cung di chuyển bốn hướng với chi phí bước đồng nhất, A* sử dụng Manhattan là phương pháp có kết quả tốt về chất lượng đường đi và số nút mở rộng ở những trường hợp được khảo sát. Tuy nhiên, việc lựa chọn thuật toán trong ứng dụng thực tế vẫn cần xét đồng thời yêu cầu tối ưu, thời gian chạy, bộ nhớ và cơ chế quản lý trạng thái. Kết quả này là cơ sở cho Chương 6, nơi ảnh hưởng của heuristic được phân tích sâu hơn thông qua các bản đồ giá trị h(n) và f(n).

CHƯƠNG 6: Ảnh hưởng của heuristic

6.1. Vai trò của heuristic trong bài toán tìm kiếm

Trong các phương pháp tìm kiếm có thông tin (Informed Search), heuristic là hàm ước lượng chi phí còn lại từ một trạng thái đang xét đến trạng thái đích. Thông tin này giúp thuật toán đánh giá mức độ triển vọng của những trạng thái trong frontier mà không cần tìm trước toàn bộ đường đi. Khác với các phương pháp tìm kiếm không thông tin như BFS, GBFS và A* có thể khai thác heuristic để tập trung tìm kiếm vào những vùng được đánh giá là có khả năng dẫn đến đích.

Gọi n là trạng thái hiện tại, G là trạng thái đích và h(n) là chi phí ước lượng từ n đến G. GBFS ưu tiên nút có h(n) nhỏ nhất, trong khi A* sử dụng hàm đánh giá f(n) = g(n) + h(n), với g(n) là chi phí đã tích lũy từ trạng thái bắt đầu đến n. Vì vậy, chất lượng heuristic ảnh hưởng trực tiếp đến thứ tự mở rộng nút, lượng công việc tìm kiếm và, tùy thuật toán, chất lượng đường đi được trả về.

Bài thực hành sử dụng notebook 06_Explore_heuristics.ipynb để quan sát bản đồ h(n), f(n) và so sánh các thuật toán trên mê cung rỗng 10 × 10. Trạng thái bắt đầu là S = (1, 1), trạng thái đích G = (8, 8), với chỉ số ma trận NumPy bắt đầu từ 0; nhãn tọa độ trên hình được hiển thị từ 1. Tác tử di chuyển theo bốn hướng lên, phải, xuống, trái và mỗi bước hợp lệ có chi phí bằng 1. Ngoài các ví dụ minh họa của notebook gốc, phần thực nghiệm bổ sung chạy BFS, GBFS và A* bằng ba heuristic, đồng thời khảo sát small_maze có tường để đối chiếu với môi trường không vật cản.

6.2. Khảo sát giá trị heuristic

6.2.1. Heuristic Manhattan

Khoảng cách Manhattan, còn được gọi là chuẩn L_1, được xác định bằng tổng độ chênh lệch tuyệt đối theo hai trục tọa độ. Với n = (r, c) và G = (rG, cG), hàm heuristic có dạng:

$$hM(n) = |r - rG| + |c - cG|$$

Mỗi hành động trong bài toán chỉ thay đổi một đơn vị ở hàng hoặc cột nên, ngay cả khi không có tường, tác tử phải thực hiện ít nhất $|r - rG|$ bước theo hàng và $|c - cG|$ bước theo cột. Do đó, Manhattan phản ánh chính xác chi phí còn lại trên lưới rỗng di chuyển bốn hướng, tức $hM(n) = h^*(n)$, với $h^*(n)$ là chi phí tối ưu thực sự từ n đến đích. Đây là một heuristic hoàn hảo (perfect heuristic) trong đúng điều kiện không có vật cản và chi phí bước bằng 1, nhưng không nhất thiết bằng $h^*(n)$ khi mê cung có tường.

Tại trạng thái bắt đầu $S = (1, 1)$, độ chênh lệch hàng và cột so với $G = (8, 8)$ đều bằng 7. Vì vậy, $hM(S) = 7 + 7 = 14$. Giá trị heuristic giảm khi tác tử di chuyển theo hướng rút ngắn khoảng cách hàng hoặc cột đến G. Trong bản đồ h(n) của notebook, các vùng có giá trị nhỏ tập trung gần đích, trong khi các vị trí xa đích có giá trị lớn hơn.

6.2.2. Heuristic Euclidean

Khoảng cách Euclidean, tương ứng với chuẩn L_2, biểu diễn độ dài đoạn thẳng nối trạng thái hiện tại và trạng thái đích. Công thức được xác định như sau:

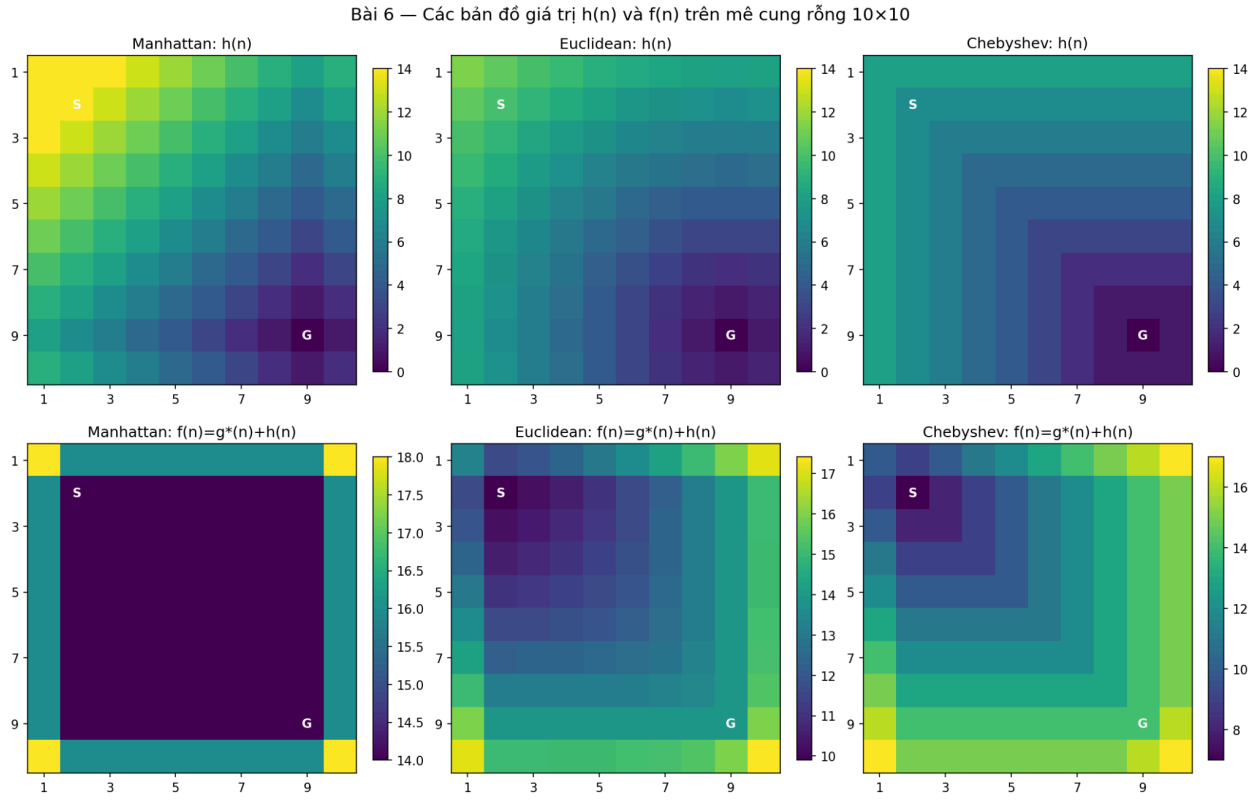
$$hE(n) = \sqrt{[(r - rG)^2 + (c - cG)^2]}$$

Với $S = (1, 1)$ và $G = (8, 8)$, ta có $hE(S) = \sqrt{(7^2 + 7^2)} = \sqrt{98} \approx 9{,}8995$. Giá trị này nhỏ hơn Manhattan tại cùng trạng thái, mặc dù chi phí đường đi ngắn nhất thực tế trong lưới rỗng là 14 bước. Nguyên nhân là Euclidean đo khoảng cách đường thẳng, còn tác tử không thể di chuyển chéo mà phải hoàn thành các bước theo từng trục.

Đối với bài toán bốn hướng có chi phí mỗi bước bằng 1, Euclidean không đánh giá vượt chi phí tối ưu thực tế và vẫn là heuristic hợp lệ. Tuy nhiên, so với Manhattan, Euclidean thường cung cấp ước lượng ít chặt hơn vì không phản ánh trực tiếp số bước bắt buộc theo hàng và cột. Sự khác biệt này có thể khiến A* phải mở rộng nhiều trạng thái hơn trước khi tìm được đích, tùy vào cách quản lý frontier và quy tắc xử lý các nút cùng mức ưu tiên.

6.2.3. Quan sát bản đồ $h(n)$ và $f(n)$

Notebook trực quan hóa bản đồ giá trị h(n) của Manhattan, Euclidean và heuristic Chebyshev được bổ sung để so sánh. Bản đồ thứ hai thể hiện f(n) = g*(n) + h(n), trong đó g*(n) là chi phí ngắn nhất từ S đến n trên lưới rỗng. Vì không có vật cản, g*(n) được tính bằng khoảng cách Manhattan từ S đến n. Đây là bản đồ đánh giá lý thuyết cho toàn bộ vị trí, không phải toàn bộ các giá trị g(n) phát sinh trong một lần chạy A* cụ thể.



Hình 6.1. Bản đồ giá trị h(n) và f(n) của Manhattan, Euclidean và Chebyshev trên lưới rỗng 10 × 10.

Đối với Manhattan, giá trị f(n) bằng 14 tại các vị trí thuộc hình chữ nhật tọa độ nằm giữa S và G, bao gồm hai đầu mút. Trên ma trận từ (1, 1) đến (8, 8), có 64 vị trí như vậy. Đây là một vùng đồng mức (plateau) có f(n) bằng chi phí tối ưu C* = 14. Bởi vậy, dù Manhattan là perfect heuristic trong mê cung rỗng, quy tắc chọn nút khi các giá trị f(n) bằng nhau vẫn có ảnh hưởng đến số nút được mở rộng.

Với Euclidean, h(n) giảm theo khoảng cách đường thẳng đến G, nhưng các giá trị f(n) không tạo thành vùng đồng mức rộng tương tự Manhattan. Với Chebyshev, sự ước lượng còn thấp hơn hoặc bằng Euclidean nên có thể dẫn đến việc nhiều trạng thái được coi là có triển vọng. Kết quả này cho thấy bản đồ h(n) không đủ để đánh giá hiệu quả A* nếu không xem xét đồng thời chi phí đã đi g(n) và quy tắc quản lý frontier.

Để minh họa, Bảng 6.1 tổng hợp giá trị heuristic tại một số trạng thái. Tất cả các vị trí đều tính theo chỉ số mảng bắt đầu từ 0.

Bảng 6.1. Giá trị của các heuristic tại một số trạng thái trong mê cung rỗng

Trạng thái	g*(n)	Manhattan hM(n)	Euclidean hE(n)	Chebyshev hC(n)
S = (1, 1)	0	14	9,8995	7
(4, 4)	6	8	5,6569	4
(1, 8)	7	7	7	7
G = (8, 8)	14	0	0	0

Tại (4, 4), giá trị f(n) tương ứng là 14 đối với Manhattan, khoảng 11,6569 đối với Euclidean và 10 đối với Chebyshev. Các khác biệt này bắt nguồn từ mức độ ước lượng khác nhau của từng heuristic, mặc dù tất cả cùng sử dụng một trạng thái và một mô hình chi phí.

6.3. So sánh hiệu quả heuristic đối với GBFS và A*

Để so sánh trực tiếp các heuristic, phần code bổ sung chạy BFS làm mốc đối chiếu và áp dụng Manhattan, Euclidean, Chebyshev cho GBFS và A*. A* lựa chọn nút theo giá trị f(n) = g(n) + h(n); nếu hai nút có cùng f(n), chương trình ưu tiên nút có h(n) nhỏ hơn, sau đó xét thứ tự đưa nút vào frontier. GBFS ưu tiên trực tiếp theo h(n). Chỉ số Nodes expanded đếm số nút được lấy ra để sinh trạng thái kế tiếp, không bao gồm nút đích; Max frontier là kích thước lớn nhất của danh sách chờ theo quy ước cài đặt này.

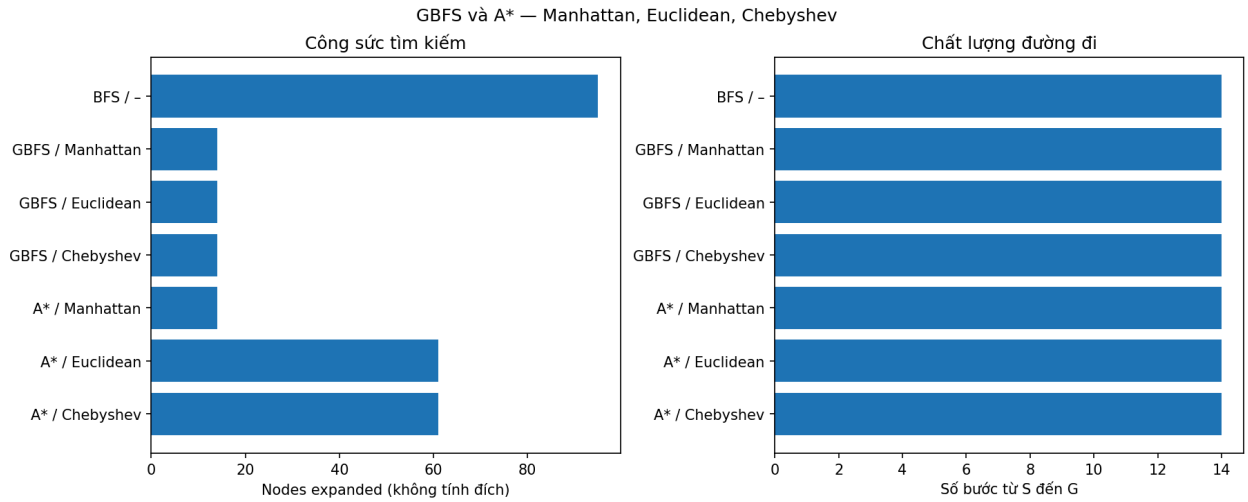
Bảng 6.2 trình bày kết quả trên mê cung rỗng 10 × 10, trong đó đường đi ngắn nhất từ S đến G có 14 bước.

Bảng 6.2. So sánh BFS, GBFS và A bằng ba heuristic trên mê cung rỗng*

Thuật toán	Heuristic	Path cost	Nodes expanded	Max frontier
BFS	Không sử dụng	14	95	12
GBFS	Manhattan	14	14	29
GBFS	Euclidean	14	14	18
GBFS	Chebyshev	14	14	18
A*	Manhattan	14	14	29

A*	Euclidean	14	61	20
A*	Chebyshev	14	61	20

Cả ba phiên bản GBFS đều tìm được đường đi 14 bước và mở rộng 14 nút. Tuy nhiên, đây chỉ là kết quả của mê cung rỗng, nơi không có tường buộc tác tử đi vòng; không thể suy ra GBFS bảo đảm tối ưu trong mọi mê cung. Đối với A*, Manhattan tìm đường 14 bước với 14 nút mở rộng, còn Euclidean và Chebyshev cũng tìm đường 14 bước nhưng đều mở rộng 61 nút. Sự chênh lệch cho thấy heuristic chặt hơn có thể giảm đáng kể số nút mở rộng khi kết hợp với quy tắc phá hòa được sử dụng trong thí nghiệm.



*Hình 6.2. So sánh số nút mở rộng và độ dài đường đi của BFS, GBFS và A** trong mê cung rỗng 10 × 10.

Đáng chú ý, Max frontier của A*–Manhattan đạt 29, cao hơn A*–Euclidean (20) và BFS (12), mặc dù A*–Manhattan mở rộng ít nút hơn. Do đó, không thể đồng nhất số nút mở rộng với nhu cầu lưu trữ; đánh giá bộ nhớ cần xét riêng cấu trúc frontier và dữ liệu trạng thái mà mỗi thuật toán duy trì. Tương tự, số nút mở rộng ít không trực tiếp chứng minh thời gian chạy ngắn hơn, vì thời gian còn phụ thuộc thao tác tính heuristic và quản lý hàng đợi ưu tiên.

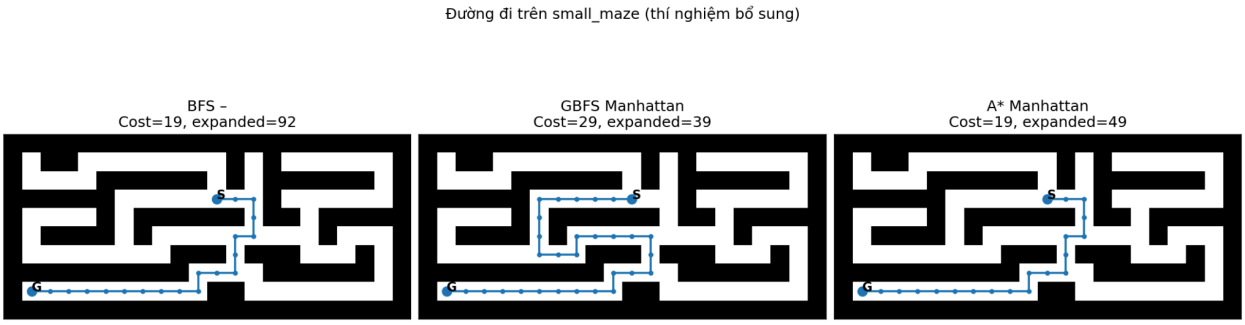
Để khảo sát ảnh hưởng của tường, phần thực nghiệm bổ sung sử dụng small_maze.txt với đường đi ngắn nhất dài 19 bước. Bảng 6.3 trình bày các kết quả thu được từ cùng chương trình thực nghiệm của Bài 6.

*Bảng 6.3. So sánh BFS, GBFS và A** trên small_maze có tường

Thuật toán	Heuristic	Path cost	Nodes expanded	Max frontier
BFS	Không sử dụng	19	92	9
GBFS	Manhattan	29	39	5
GBFS	Euclidean	29	39	5
GBFS	Chebyshev	29	54	4
A*	Manhattan	19	49	8
A*	Euclidean	19	56	7
A*	Chebyshev	19	56	6

Trên small_maze, GBFS–Manhattan và GBFS–Euclidean đều chỉ mở rộng 39 nút nhưng tìm đường đi 29 bước, dài hơn đường tối ưu 10 bước. GBFS–Chebyshev cũng trả về đường 29 bước với 54 nút mở rộng. Trong khi đó, A* với cả ba heuristic đều giữ chi phí tối ưu bằng 19; A*–Manhattan mở rộng 49 nút, thấp hơn

A*–Euclidean và A*–Chebyshev (56 nút). Kết quả củng cố nhận xét rằng việc chỉ ưu tiên khoảng cách ước lượng có thể dẫn đến đường đi chưa tối ưu khi xuất hiện vật cản.



Hình 6.3. Đường đi của BFS, GBFS–Manhattan và A–Manhattan trên small_maze có tường.

Kết quả A*–Manhattan ở thí nghiệm này là 49 nút mở rộng, khác giá trị 51 của phần khảo sát Bài 5. Sự khác biệt đến từ quy tắc ưu tiên khi nhiều nút có cùng giá trị f(n) và cách quản lý trạng thái của hai phiên bản thực nghiệm; vì vậy, các con số chỉ được so sánh trực tiếp giữa những lần chạy cùng một cài đặt. Kết quả về tính tối ưu của đường đi vẫn nhất quán: A* tìm được 19 bước trong cả hai bài.

6.4. Đề xuất heuristic thay thế

6.4.1. Heuristic Chebyshev

Ngoài Manhattan và Euclidean, bài thực hành đề xuất sử dụng khoảng cách Chebyshev, còn gọi là chuẩn L∞, để đánh giá trạng thái. Với n = (r, c) và G = (rG, cG), biểu thức được xác định như sau:

$$h_C(n) = \max(|r - r_G|, |c - c_G|)$$

Chebyshev chỉ lấy độ lệch lớn nhất trên hai trục tọa độ nên thường được sử dụng làm khoảng cách phù hợp cho môi trường cho phép di chuyển ngang, dọc và chéo với cùng chi phí bước. Trong mê cung bốn hướng của Lab 02, nó không biểu diễn

chính xác số bước tối thiểu khi cần thay đổi đồng thời cả hàng và cột, nhưng vẫn có thể dùng làm một ước lượng không vượt chi phí thực.

Tại S = (1, 1), G = (8, 8), Chebyshev cho hC(S) = max(7, 7) = 7, nhỏ hơn Manhattan là 14 và Euclidean khoảng 9,8995. Trên lưới rỗng có bốn hướng, ba heuristic thỏa mãn quan hệ $hC(n) \leq hE(n) \leq hM(n) = h^*(n)$. Do đó, Chebyshev kém thông tin hơn Manhattan trong mô hình này, nhưng vẫn hữu ích làm đối chứng để quan sát ảnh hưởng của mức độ ước lượng đến hiệu suất tìm kiếm.

Kết quả Bảng 6.2 cho thấy A*–Chebyshev tìm được đường 14 bước nhưng mở rộng 61 nút, nhiều hơn đáng kể so với 14 nút của A*–Manhattan. Trên small_maze, A*–Chebyshev vẫn tìm được lời giải tối ưu 19 bước nhưng mở rộng 56 nút thay vì 49 nút của Manhattan. Những kết quả này phù hợp với nhận xét rằng cận dưới chặt hơn có thể cung cấp định hướng tốt hơn cho A* trong bài toán bốn hướng.

6.4.2. Điều kiện admissibility và consistency

Một heuristic được gọi là admissible (không đánh giá vượt) khi $h(n) \leq h^*(n)$ đối với mọi trạng thái n có thể đi đến đích, trong đó $h^*(n)$ là chi phí tối ưu thực tế còn lại. Trong bài toán tìm đường có chi phí không âm, heuristic admissible không khiến thuật toán đánh giá quá cao chi phí cần thiết để đến đích. Đặc biệt, với lưới bốn hướng và chi phí bước 1, Manhattan, Euclidean và Chebyshev đều admissible vì mỗi giá trị không vượt số bước tối ưu thực tế, kể cả khi có tường.

Consistency (tính nhất quán) là điều kiện mạnh hơn, đòi hỏi $h(n) \leq c(n, n') + h(n')$ đối với mọi chuyển trạng thái hợp lệ từ n sang n', đồng thời $h(G) = 0$. Trong mê cung đang xét, $c(n, n') = 1$ cho mỗi bước đi hợp lệ. Manhattan, Euclidean và Chebyshev đều thỏa điều kiện này bởi mỗi bước di chuyển chỉ thay đổi một tọa độ một đơn vị, nên khoảng cách được ước lượng không thể giảm nhiều hơn chi phí của một bước.

Với Chebyshev, nếu đặt $a = |r - rG|$ và $b = |c - cG|$ thì $hC(n) = \max(a, b) \leq a + b = hM(n) \leq h^*(n)$. Điều này chứng minh tính admissible. Đối với hai trạng thái kề nhau n và n', độ lệch của mỗi tọa độ chỉ thay đổi tối đa một đơn vị, suy ra $hC(n) \leq 1 + hC(n')$. Vì thế, Chebyshev cũng consistent trong mô hình bốn hướng, mỗi bước có chi phí 1.

Đối với A* tìm kiếm trên cây, heuristic admissible là điều kiện quan trọng để bảo đảm tối ưu trong các giả định tìm kiếm phù hợp. Khi triển khai A* trên đồ thị có tập reached hoặc closed, heuristic consistent cho phép dùng cơ chế đóng trạng thái đã mở rộng mà vẫn bảo đảm tối ưu nếu thuật toán quản lý chi phí đúng. Nếu heuristic chỉ admissible nhưng không consistent, thuật toán có thể phải mở lại trạng thái khi phát hiện một đường đi tốt hơn. Vì vậy, điều kiện tối ưu không chỉ phụ thuộc công thức h(n) mà còn phụ thuộc cách tổ chức frontier, kiểm tra trùng trạng thái và cập nhật chi phí.

Trong notebook, các phép kiểm tra trên toàn bộ lưới rỗng đã xác nhận cả ba heuristic không âm, bằng 0 tại đích, không vượt chi phí tối ưu còn lại và thỏa bất đẳng thức nhất quán đối với các ô kề nhau. Đây là minh chứng tính đúng đắn trong miền dữ liệu thực nghiệm; phần lập luận ở trên giải thích vì sao các tính chất này còn áp dụng cho mô hình lưới bốn hướng với chi phí đồng nhất.

6.5. Nhận xét

Qua khảo sát, các bản đồ h(n) và f(n) cho thấy việc lựa chọn heuristic quyết định cách thuật toán đánh giá những trạng thái có khả năng dẫn đến đích. Manhattan phản ánh đúng số bước tối thiểu còn lại trong mê cung rỗng bốn hướng; Euclidean biểu diễn khoảng cách đường thẳng và Chebyshev lấy độ lệch lớn nhất theo một trục. Trong mô hình thực nghiệm, Manhattan là ước lượng chặt nhất trong ba heuristic và cung cấp khả năng định hướng rõ hơn cho A*.

Trên lưới rỗng 10 × 10, tất cả các thuật toán được khảo sát đều tìm được đường đi dài 14 bước. Tuy nhiên, A*–Manhattan chỉ mở rộng 14 nút trong khi A*–Euclidean và A*–Chebyshev mỗi thuật toán mở rộng 61 nút. Sự chênh lệch này phản ánh ảnh hưởng của heuristic đến số nút được mở rộng, đồng thời chịu tác động của quy tắc phá hòa giữa các nút có cùng giá trị f(n). Ngoài ra, Max frontier của A*–Manhattan cao hơn các trường hợp còn lại, chứng tỏ số nút mở rộng thấp chưa đủ để kết luận nhu cầu bộ nhớ nhỏ.

Thí nghiệm trên small_maze có tường làm rõ hơn sự khác biệt giữa GBFS và A*. GBFS sử dụng các heuristic khảo sát đều tìm đường 29 bước, trong khi BFS và A* tìm được đường tối ưu 19 bước. A*–Manhattan chỉ mở rộng 49 nút so với 92 nút

của BFS trong cùng thiết lập. Điều này cho thấy kết hợp chi phí đã đi với heuristic phù hợp có thể cải thiện hiệu quả mở rộng trạng thái mà vẫn duy trì chất lượng lời giải.

Heuristic Chebyshev được đề xuất thêm đã thỏa các điều kiện admissibility và consistency trong mô hình bốn hướng, chi phí mỗi bước bằng 1. Tuy nhiên, do ước lượng kém chặt hơn Manhattan, số nút mở rộng của A*–Chebyshev lớn hơn trong các thí nghiệm. Qua đó, bài thực hành khẳng định rằng một heuristic hợp lệ không nhất thiết là heuristic hiệu quả nhất; cần xét cả độ chặt của ước lượng, cách triển khai và đặc điểm môi trường.

Nhìn chung, Manhattan là lựa chọn phù hợp trong bài toán mê cung di chuyển bốn hướng của Lab 02. Euclidean và Chebyshev vẫn bảo đảm các điều kiện lý thuyết để dùng trong A* nhưng có thể đòi hỏi nhiều lần mở rộng hơn. Kết quả của chương cung cấp cơ sở để hiểu sâu hơn sự tương tác giữa heuristic, quản lý frontier và chất lượng lời giải, đồng thời chuẩn bị cho phần khảo sát kiểm tra chu trình trong DFS ở chương tiếp theo.

CHƯƠNG 7: Kiểm tra chu trình trong DFS

7.1. Vấn đề chu trình trong tìm kiếm theo chiều sâu

Tìm kiếm theo chiều sâu (Depth-First Search – DFS) ưu tiên mở rộng những trạng thái nằm sâu trên nhánh đang khảo sát trước khi quay lui để xét các lựa chọn khác. Trên bài toán mê cung, mỗi trạng thái được biểu diễn bằng tọa độ hàng và cột; tác tử có thể di chuyển theo bốn hướng hợp lệ. Khi giữa các ô có nhiều đường kết nối, một trạng thái có thể được tiếp cận qua nhiều nhánh và các bước di chuyển có thể tạo thành chu trình. Nếu không kiểm soát việc quay lại trạng thái đã có trên đường đi, thuật toán có nguy cơ liên tục tạo ra những nút không mang thêm thông tin mới.

Kiểm tra chu trình (cycle checking) là cơ chế phát hiện và loại bỏ những lựa chọn có thể dẫn đến việc lặp lại trạng thái. Tuy nhiên, cần phân biệt việc lặp lại một trạng thái trên đường đi hiện tại với việc cùng một trạng thái xuất hiện ở nhiều nhánh của cây tìm kiếm. Hai hiện tượng này không tương đương: ngăn một đường đi tự quay vòng chưa chắc ngăn được một trạng thái được đưa vào danh sách chờ (frontier) nhiều lần hoặc bị mở rộng lại ở một nhánh khác.

Trong notebook 05_Maze_Example-Animation_Cycle_checking.ipynb, ba phương án được minh họa bằng những cách phối hợp kiểm tra đường đi và quản lý ngăn xếp (stack) của DFS. Mục tiêu bài thực hành là quan sát số lần lấy nút khỏi stack, số nút thực sự được mở rộng, kích thước frontier và các biểu hiện lặp trạng thái trên cùng dữ liệu open_maze. Khác với những bài tìm đường trước tập trung vào độ dài lời giải, nội dung chương này chú trọng cơ chế quản lý trạng thái trong quá trình tìm kiếm.

7.2. Các phương án kiểm tra chu trình

7.2.1. Kiểm tra trạng thái trên đường đi hiện tại

Phương án thứ nhất chỉ kiểm tra xem trạng thái của nút vừa được lấy khỏi stack có trùng với một trạng thái tổ tiên trong đường đi từ nút gốc đến nút hiện tại hay

không. Nếu trùng, nút bị loại và thuật toán lấy phần tử tiếp theo; nếu không trùng và chưa đạt đích, nút được mở rộng để sinh các trạng thái kế tiếp. Trong notebook, phương án này tương ứng với frontier_option = 3, được mô tả bằng tên “Path only”.

Ưu điểm của cách làm là ngăn các chu trình xuất hiện trực tiếp trên một đường đi, đồng thời không cần một tập trạng thái đã thăm toàn cục để cắt bỏ mọi lần tái tiếp cận. Tuy nhiên, vì chưa kiểm tra những nút đang có trong frontier, cùng một trạng thái có thể tồn tại đồng thời dưới nhiều đối tượng Node với các nút cha khác nhau. Điều này làm tăng số phần tử phải lưu và có thể dẫn đến việc khảo sát lại cùng một vùng mê cung nhiều lần.

7.2.2. Kiểm tra đường đi và tránh thêm trạng thái đã có trong frontier

Phương án thứ hai kết hợp kiểm tra đường đi hiện tại với việc phát hiện trạng thái đã có trong stack. Khi sinh một nút con, nếu một entry mang cùng trạng thái đang chờ trong frontier, chương trình bỏ qua nút mới thay vì tạo thêm bản sao. Trong notebook, cách này tương ứng với frontier_option = 2, được gọi là “Path + no duplicate frontier”.

Cơ chế trên có thể giảm số phần tử trùng xuất hiện đồng thời trong frontier. Tuy nhiên, việc không có bản sao trong stack tại một thời điểm không bảo đảm trạng thái chỉ được mở rộng một lần trong toàn bộ quá trình. Sau khi một nút đã bị lấy khỏi stack, cùng vị trí vẫn có thể được sinh lại và đưa vào frontier từ nhánh khác. Đồng thời, thao tác bỏ qua một nút mới có thể giữ lại một entry cũ vốn thuộc nhánh đang chờ, khiến thứ tự duyệt chưa phản ánh hợp lý nhánh DFS vừa được mở rộng.

Đây cũng là vấn đề notebook gốc nhấn mạnh khi quan sát animation: những nút đang chờ có thể còn lại trong frontier sau khi một nhánh đã được xử lý, khiến thuật toán tiếp tục quay lại khảo sát vùng trước đó. Vì vậy, chỉ ngăn nút trùng trong danh sách chờ chưa đủ để hạn chế hiện tượng khám phá lại trong mọi trường hợp.

7.2.3. Kiểm tra đường đi và điều chỉnh thứ tự trong frontier

Phương án thứ ba vừa kiểm tra chu trình trên đường đi, vừa điều chỉnh vị trí của entry khi cùng một trạng thái được phát hiện lại trong frontier. Thay vì chỉ bỏ qua nút mới, chương trình xóa entry cũ, tạo hoặc cập nhật entry gắn với đường đi mới và đưa entry đó lên đỉnh stack. Với cách cài đặt stack bằng danh sách Python, đỉnh stack là cuối danh sách, nơi thao tác pop() lấy nút ra trước. Phương án này tương ứng với frontier_option = 1, được mô tả là “Path + move entry to stack top”.

Việc đưa trạng thái được tìm thấy lại lên đỉnh stack thay đổi thứ tự ưu tiên theo hướng tiếp tục nhánh đang duyệt. Trong bản cài đặt minh họa, entry mới mang thông tin parent, action và cost theo đường đi vừa sinh. Cách xử lý này duy trì nhiều thông tin hơn về quá trình chuyển trạng thái và tránh đồng thời lưu nhiều entry cho cùng một vị trí.

Đây là một lựa chọn quản lý frontier dành cho thí nghiệm, không phải quy tắc bảo đảm DFS luôn nhanh hơn hoặc tìm đường tối ưu trên mọi bài toán. Hiệu quả vẫn phụ thuộc cấu trúc mê cung, thứ tự sinh trạng thái và quy tắc cập nhật entry khi gặp lại cùng một vị trí.

7.3. Thực nghiệm trên open_maze

Dữ liệu thực nghiệm được đọc từ open_maze.txt bằng hàm parse_maze() của mô-đun maze_helper. Bản đồ có 23 hàng và 37 cột, với điểm bắt đầu S = (1, 35) và đích G = (21, 1) theo tọa độ ma trận bắt đầu từ 0. Mê cung có những vùng mở tương đối lớn và nhiều nhánh liên thông, thích hợp để quan sát hiện tượng một trạng thái được tiếp cận bằng nhiều đường đi.

Ba phương án được chạy độc lập, sử dụng cùng mê cung, thứ tự thêm hành động NESW (Bắc, Đông, Nam, Tây), chi phí bước bằng 1 và giới hạn max_tries = 500. Trong đó, một iteration là một lần lấy nút khỏi stack bằng pop(), kể cả khi nút sau đó bị loại do trùng trạng thái trên đường đi. Một nút chỉ được tính vào Nodes expanded nếu vượt qua kiểm tra chu trình, không phải đích và được xử lý để sinh trạng thái kế tiếp.

Mã tree_search_solution.py sử dụng trong bài thực hành là phiên bản cài đặt minh họa độc lập tương thích với các lựa chọn của notebook, không phải mã nguồn gốc chưa công bố của tác giả. Do đó, số liệu dưới đây phản ánh phiên bản thực nghiệm được cung cấp và có thể khác khi thay đổi cách triển khai.

Bảng 7.1. Kết quả thực nghiệm ba phương án kiểm tra chu trình trên open_maze

Chỉ số	**Phương án 1**	**Phương án 2**	**Phương án 3**
frontier_option	3	2	1
Số lần lấy nút khỏi stack	500	500	434
Số lượt mở rộng nút	221	250	211
Số trạng thái mở rộng phân biệt	179	116	211
Số lượt mở rộng lặp	42	134	0
Kích thước frontier lớn nhất	328	162	143
Số entry trùng lớn nhất trong frontier	129	0	0
Nút bị loại do chu trình	279	250	222

Nút mới bị bỏ qua vì đã ở frontier	0	231	0
Lượt đưa entry lên đỉnh stack	0	0	201
Trạng thái kết thúc	Chạm giới hạn	Chạm giới hạn	Tìm thấy đích
Chi phí đường đi	—	—	54

Kết quả cho thấy hai phương án đầu đều sử dụng hết giới hạn 500 lần lấy nút mà chưa tìm được đích. Trong khi đó, phương án thứ ba tìm được đường đi có chi phí 54 sau 434 lần lấy nút khỏi stack. Trường hợp chưa tìm thấy đích khi đạt max_tries = 500 chỉ phản ánh giới hạn của lần chạy; không được xem là bằng chứng mê cung không có đường đi hay thuật toán chắc chắn lặp vô hạn.

Bảng cũng thể hiện sự khác biệt giữa số nút thực sự mở rộng và số trạng thái phân biệt. Chẳng hạn, phương án thứ hai thực hiện 250 lượt mở rộng nhưng chỉ có 116 vị trí phân biệt được mở rộng. Điều này cho thấy tránh các bản sao trong frontier tại cùng một thời điểm chưa ngăn được việc mở rộng lại trạng thái ở những thời điểm khác nhau.

7.4. So sánh các phương án cycle checking

7.4.1. Số lần lặp và số nút mở rộng

Chỉ số iterations được tính cho mỗi lần thực hiện thao tác pop() trên stack. Đây là số lần lấy nút ra để kiểm tra, khác với Nodes expanded là số lượt nút thực sự được mở rộng. Nếu nút bị loại vì trùng một trạng thái tổ tiên, thao tác pop() vẫn được tính vào iterations nhưng không tạo thêm một lượt mở rộng. Việc phân biệt hai chỉ

số giúp tránh nhận định sai rằng mọi nút được lấy khỏi frontier đều làm phát sinh các nút con.

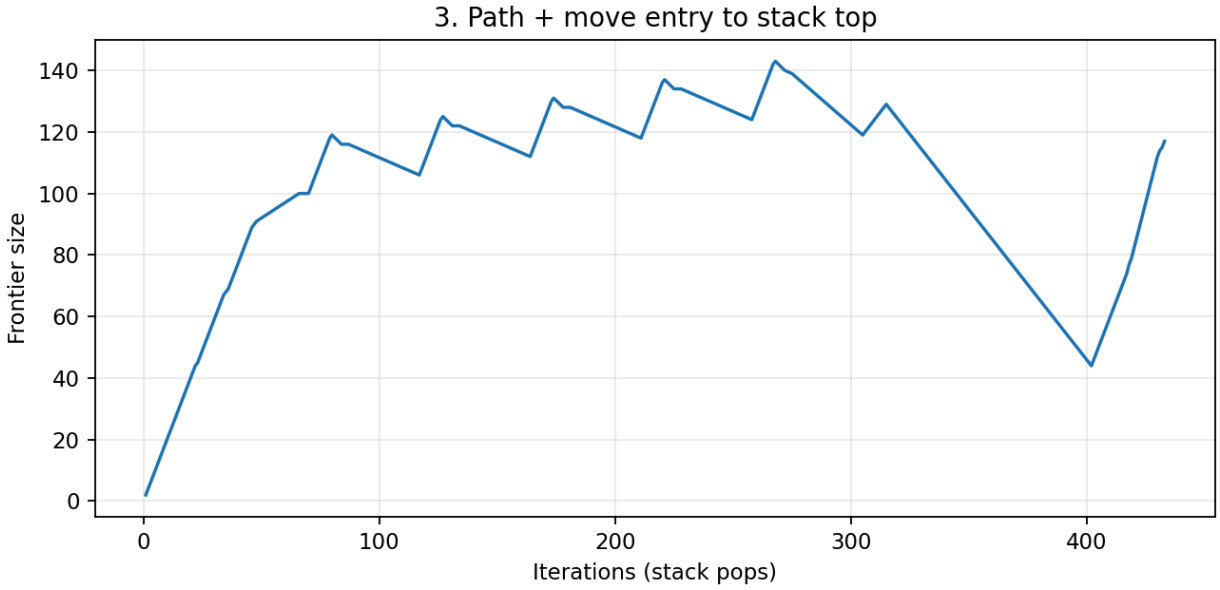
Đối với phương án thứ nhất, 500 lượt pop() tạo ra 221 lượt mở rộng và 279 lượt bị loại do kiểm tra chu trình. Trong số 221 lượt mở rộng, chỉ có 179 trạng thái phân biệt, nghĩa là có 42 lượt mở rộng lặp. Phương án thứ hai có 250 lượt mở rộng, 250 lượt bị loại do chu trình nhưng chỉ có 116 trạng thái phân biệt; số lượt mở rộng lặp tăng lên 134. Kết quả này chỉ ra rằng kiểm soát trùng frontier đơn thuần chưa giải quyết được hiện tượng một trạng thái được duyệt lại sau khi entry cũ đã rời stack.

Ở phương án thứ ba, chương trình kết thúc sau 434 lượt pop(), gồm 211 lượt mở rộng, 222 lượt bị loại do chu trình và một lượt lấy ra nút đích. Cả 211 lượt mở rộng ứng với 211 trạng thái phân biệt trong lần chạy này, tức không ghi nhận lượt mở rộng lặp. Trong cùng giới hạn khảo sát, cách điều chỉnh vị trí entry giúp thuật toán tiến đến đích thay vì tiếp tục tái khám phá những trạng thái đã xét.

7.4.2. Kích thước frontier

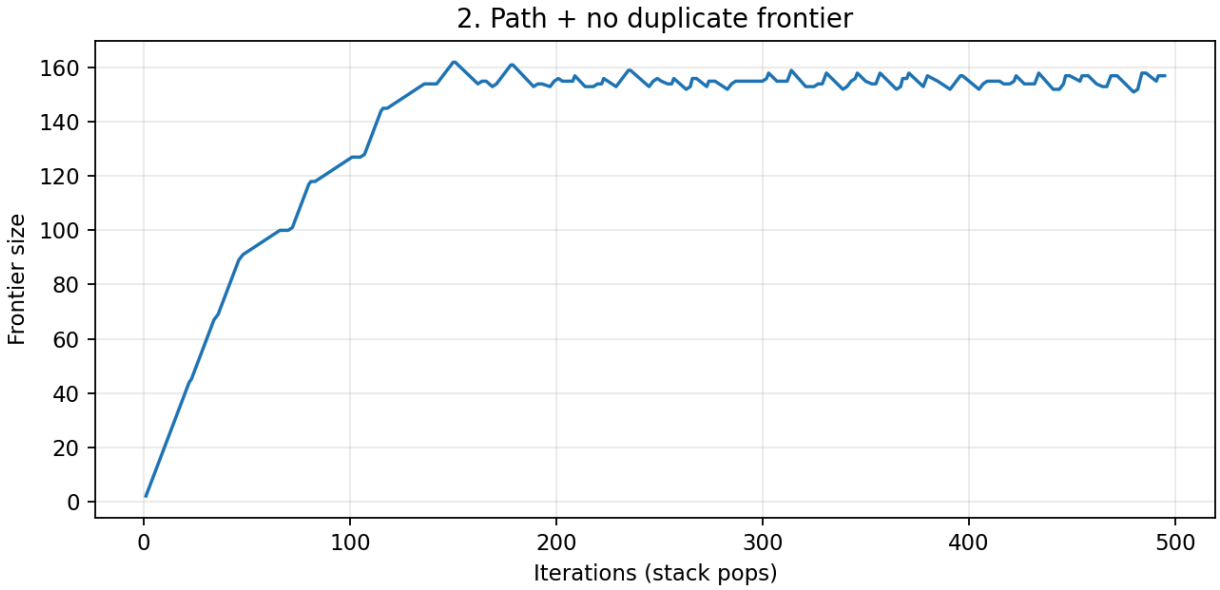
Frontier biểu diễn tập các nút đã được sinh nhưng đang chờ mở rộng. Kích thước frontier có thể tăng khi một nút sinh nhiều trạng thái kế tiếp, và giảm khi các nút bị lấy ra hoặc không thể sinh thêm nhánh hợp lệ. Chỉ số Max frontier ghi nhận số entry lớn nhất tại một thời điểm, không phải tổng số nút đã tạo ra và cũng không đại diện đầy đủ cho bộ nhớ chương trình tính bằng byte.

Ở phương án thứ nhất, frontier tăng nhanh trong các lượt đầu, sau đó dao động quanh vùng có hơn 300 entry. Kích thước lớn nhất đạt 328, đi kèm tối đa 129 entry bản sao dư thừa tồn tại đồng thời trong stack. Diễn biến này phù hợp với việc không loại bỏ trạng thái trùng khi đưa vào frontier.



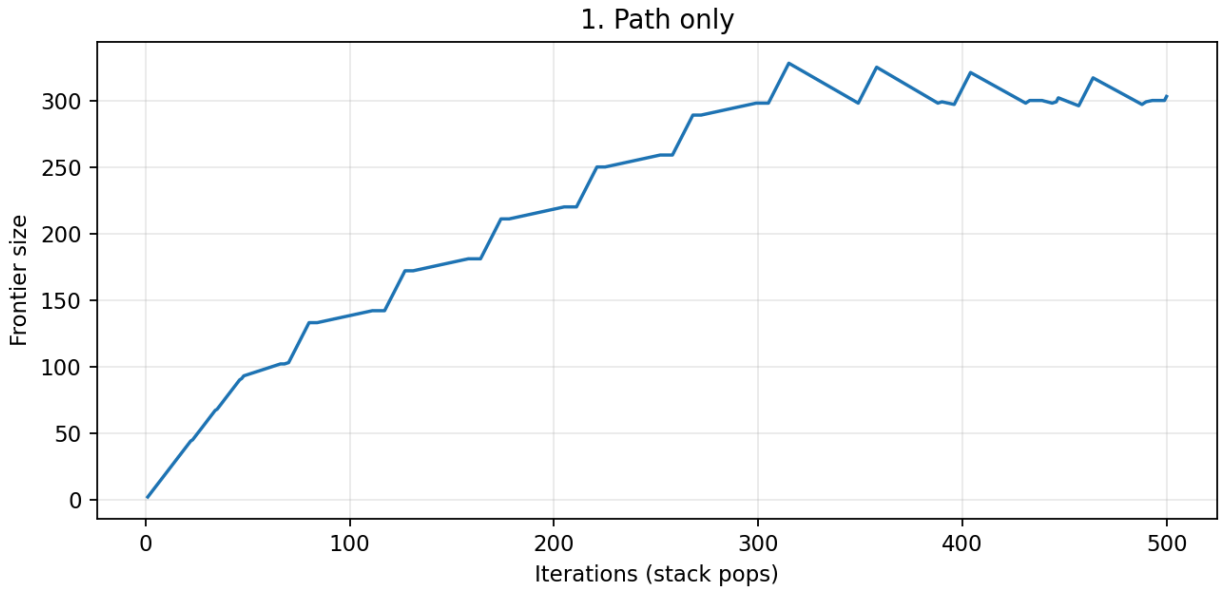
Hình 7.1. Biến động kích thước frontier của phương án 1 – chỉ kiểm tra đường đi hiện tại.

Đối với phương án thứ hai, kích thước frontier tăng đến tối đa 162 rồi dao động quanh mức 150–160 entry trong phần lớn giai đoạn sau. Nhờ từ chối trạng thái đã có trong frontier, không xuất hiện entry trùng đồng thời trong stack. Tuy nhiên, số lượt mở rộng lặp vẫn cao, cho thấy một trạng thái có thể được đưa vào frontier trở lại sau khi entry trước đó bị lấy ra.



Hình 7.2. Biến động kích thước frontier của phương án 2 – kiểm tra đường đi và tránh trùng frontier.

Ở phương án thứ ba, frontier đạt mức lớn nhất 143 rồi biến động theo quá trình đẩy và lấy nút, bao gồm cả những thời điểm giảm mạnh khi nhiều entry đã được xử lý. Khi đích được tìm thấy, frontier chưa nhất thiết rỗng; điều kiện dừng chỉ đòi hỏi trạng thái đích đã được lấy ra và kiểm tra thành công. Kết quả quan sát cho thấy phương án này có kích thước frontier tối đa thấp nhất trong ba cách triển khai được khảo sát.



Hình 7.3. Biến động kích thước frontier của phương án 3 – chuyển entry được tìm thấy lại lên đỉnh stack.

Các đường đồ thị được ghi từ nhật ký trace sau mỗi lần mở rộng. Những lượt pop() bị loại do chu trình không tạo điểm dữ liệu mới trên đồ thị, nên đường biểu diễn thể hiện sự biến đổi frontier tại những thời điểm có bản ghi mở rộng; đây không phải đồ thị ghi đủ mọi lần pop().

7.4.3. Hiện tượng lặp trạng thái

Cần phân biệt hai chỉ số: Max duplicate entries là số bản sao dư thừa lớn nhất cùng tồn tại trong frontier tại một thời điểm; Repeated expansions là số lượt mở rộng

vượt quá số trạng thái phân biệt đã được mở rộng trong toàn bộ lần chạy. Một phương án có thể đạt Max duplicate entries bằng 0 nhưng vẫn có Repeated expansions lớn hơn 0, vì trạng thái có thể rời frontier, được mở rộng và sau đó được sinh lại từ một nhánh khác.

Phương án thứ nhất có cả hai hiện tượng: tối đa 129 entry trùng trong frontier và 42 lượt mở rộng lặp. Phương án thứ hai loại bỏ được bản sao đồng thời trong stack, nhưng lại ghi nhận 134 lượt mở rộng lặp; đây là ví dụ rõ nhất cho sự khác biệt giữa kiểm tra trùng trong frontier và tránh tái khám phá trạng thái trên toàn bộ quá trình. Phương án thứ ba không ghi nhận hai hiện tượng này trong giới hạn thực nghiệm: Max duplicate entries bằng 0 và Repeated expansions bằng 0.

Ngoài ra, phương án thứ hai bỏ qua 231 nút mới vì trạng thái đã có trong frontier; phương án thứ ba thực hiện 201 lượt chuyển entry lên đỉnh stack. Các con số này phản ánh hai cơ chế xử lý khác nhau đối với trạng thái đã chờ: một bên giữ entry cũ và bỏ qua entry mới, bên còn lại cập nhật thứ tự ưu tiên để tiếp tục nhánh đang tìm kiếm. Kết quả của một lần chạy chưa chứng minh một cơ chế sẽ luôn trội hơn trong mọi mê cung, nhưng đủ để minh họa tác động lớn của chính sách quản lý frontier đến hành vi DFS.

7.5. Nhận xét

Qua thực nghiệm trên open_maze, việc chỉ kiểm tra chu trình trên đường đi hiện tại giúp DFS tránh quay lại trạng thái tổ tiên nhưng không ngăn được nhiều bản sao cùng tồn tại trong frontier. Phương án thứ nhất có kích thước frontier lớn nhất bằng 328 và ghi nhận 42 lượt mở rộng lặp. Điều này cho thấy quản lý chu trình ở phạm vi một nhánh chưa đủ để kiểm soát hiệu quả những trạng thái có thể tiếp cận từ nhiều nhánh khác nhau.

Phương án thứ hai cải thiện tình trạng bản sao đồng thời trong frontier và giảm kích thước frontier lớn nhất xuống 162. Tuy nhiên, số lượt mở rộng lặp tăng lên 134, cao hơn phương án thứ nhất. Vì vậy, việc loại trùng ở thời điểm thêm nút vào stack không đồng nghĩa với loại bỏ tái khám phá trên toàn bộ quá trình. Cơ chế này còn chịu ảnh hưởng của cách giữ lại entry cũ thuộc nhánh chờ, trong khi nút mới phát hiện từ nhánh hiện tại bị bỏ qua.

Phương án thứ ba điều chỉnh thứ tự frontier bằng cách chuyển trạng thái được tìm thấy lại lên đỉnh stack. Trong lần khảo sát, phương án này mở rộng 211 nút tương ứng với 211 trạng thái phân biệt, đạt max_frontier bằng 143 và tìm được đích với đường đi 54 bước sau 434 lượt pop(). Kết quả cho thấy cách phối hợp kiểm tra đường đi với quản lý thứ tự entry có thể cải thiện hiệu quả khảo sát trong cấu trúc mê cung này. Tuy nhiên, DFS với cơ chế trên vẫn không có bảo đảm chung về tính tối ưu của đường đi.

Bài thực hành đồng thời làm rõ ý nghĩa của việc lựa chọn chỉ số đánh giá. Số lần pop(), số lần mở rộng, số trạng thái phân biệt, số entry trùng và max_frontier phản ánh những khía cạnh khác nhau của quá trình tìm kiếm; không thể thay thế các chỉ số này cho nhau. Bên cạnh đó, việc hai phương án dừng ở giới hạn 500 lần lặp chỉ cho thấy chưa đạt đích trong phạm vi thử nghiệm, chứ không chứng minh thuật toán vô nghiệm hoặc thực sự lặp vô hạn.

Từ những kết quả trên, có thể kết luận rằng cycle checking là thành phần quan trọng trong việc triển khai DFS trên không gian trạng thái có nhiều nhánh và chu trình. Hiệu quả của thuật toán không chỉ phụ thuộc vào việc phát hiện trạng thái lặp mà còn phụ thuộc vào cách tổ chức frontier và cập nhật quan hệ giữa các nút. Đây là cơ sở để xây dựng Goal-based Planning Agent ở Chương 8, nơi các thuật toán cần quản lý nhất quán cây tìm kiếm, frontier và trạng thái đã khảo sát.

CHƯƠNG 8: Bài tập tổng hợp – Goal-based Planning Agent

8.1. Giới thiệu Goal-based Planning Agent

Bài thực hành tổng hợp của Lab 02 tập trung xây dựng tác tử lập kế hoạch hướng mục tiêu (Goal-based Planning Agent) để giải bài toán tìm đường trong mê cung. Khác với tác tử chỉ phản ứng theo trạng thái hiện tại hoặc lựa chọn hành động ngẫu nhiên, tác tử hướng mục tiêu sử dụng thông tin về môi trường, trạng thái bắt đầu và trạng thái đích để tìm một chuỗi hành động trước khi thực thi. Vì vậy, kết quả của quá trình tìm kiếm không chỉ là xác nhận sự tồn tại của đường đi mà còn là một kế hoạch có thể kiểm tra và thực hiện được.

Notebook Ex1_Maze.ipynb được sử dụng làm bài thực hành chính, bao gồm xác định bài toán, cài đặt BFS, DFS, GBFS, A*, thực nghiệm trên tám mê cung và các phần mở rộng IDS, nhiều đích. Notebook Ex2_Maze_simplified.ipynb tập trung vào BFS để kiểm tra riêng các bước mô hình hóa, quản lý hàng đợi và đánh giá kết quả. Việc sử dụng song song hai notebook giúp đối chiếu lời giải của thuật toán với cơ chế thực thi kế hoạch của tác tử.

8.1.1. Mục tiêu và chức năng của tác tử

Mục tiêu của tác tử là đi từ ô bắt đầu S đến ô đích G bằng các hành động hợp lệ, không đi xuyên qua tường và không vượt khỏi phạm vi bản đồ. Mê cung được biểu diễn bằng ma trận ký tự: X là tường; khoảng trắng là ô có thể di chuyển; S và G lần lượt biểu diễn điểm bắt đầu và đích. Tác tử được giả định biết trước bản đồ, quá trình chuyển trạng thái là xác định và chi phí của mỗi bước hợp lệ bằng 1.

Trong Ex1_Maze.ipynb, lớp Planning_Agent tiếp nhận bốn thông tin đầu vào: maze, start, goal và planning_function. Thuộc tính plan lưu chuỗi hành động đã được lập, còn progress ghi nhận vị trí hành động hiện tại trong chuỗi. Ở lần gọi

act() đầu tiên, tác tử sử dụng planning_function để xây dựng kế hoạch; những lần gọi tiếp theo lần lượt trả về từng hành động trong plan cho môi trường thực hiện. Ví dụ ban đầu của notebook sử dụng hàm plan_random() để minh họa cơ chế hoạt động nhưng không bảo đảm giải được mê cung. Trong bài hoàn thiện, các hàm lập kế hoạch dựa trên BFS, DFS, GBFS và A* thay thế chức năng chọn ngẫu nhiên bằng chuỗi hành động được tìm từ cây tìm kiếm.

Chương trình đồng thời kiểm tra tính hợp lệ của kế hoạch bằng cách mô phỏng lại chuỗi hành động từ trạng thái bắt đầu. Một kế hoạch chỉ được chấp nhận khi mọi bước đều hợp lệ và trạng thái cuối cùng trùng với một đích cho trước. Việc kiểm tra này giúp tách biệt hai nhiệm vụ: tìm kiếm kế hoạch và xác minh rằng kế hoạch thực sự thực hiện được trong môi trường.

8.1.2. Quy trình từ trạng thái đầu đến trạng thái đích

Quy trình của tác tử bắt đầu bằng việc đọc tệp mê cung, chuyển dữ liệu thành mảng hai chiều và xác định tọa độ S, G. Sau đó, chương trình khởi tạo nút gốc tại S, lựa chọn chiến lược tìm kiếm và quản lý những nút chưa mở rộng trong frontier. Tùy thuật toán, nút được lấy ra theo nguyên tắc FIFO, LIFO hoặc giá trị ưu tiên heuristic. Khi một trạng thái đích được xác định, liên kết parent của các nút cho phép truy hồi đường đi từ S đến G và trích xuất danh sách các hành động tương ứng.

Danh sách hành động trả về là đầu ra trực tiếp mà Planning_Agent có thể sử dụng. Nếu thuật toán không tìm được lời giải trước giới hạn tài nguyên, kết quả tìm kiếm phải được đánh dấu riêng thay vì tự suy diễn rằng bản đồ vô nghiệm. Do đó, mỗi lượt thực nghiệm trả về cả trạng thái kết thúc, đường đi nếu có và những chỉ số phản ánh công sức tìm kiếm. Quy trình này tạo cơ sở để so sánh chất lượng kế hoạch giữa các thuật toán mà vẫn giữ nguyên một mô hình môi trường chung.

8.2. Thiết kế và triển khai chương trình

8.2.1. Biểu diễn trạng thái và lớp Node

Mỗi trạng thái được biểu diễn bằng cặp tọa độ (r, c), trong đó r là chỉ số hàng và c là chỉ số cột của ma trận mê cung. Bốn hành động được ký hiệu N, E, S, W, tương ứng với dịch chuyển lên, phải, xuống và trái. Một hành động chỉ hợp lệ nếu ô kế tiếp nằm trong phạm vi ma trận và không có ký hiệu X.

Lớp Node trong Ex1_Maze.ipynb có bốn thuộc tính: pos, parent, action và cost. Trong đó, pos xác định vị trí của nút; parent lưu tham chiếu đến nút cha; action ghi lại hành động đã tạo nút; cost là chi phí tích lũy tính từ S. Với chi phí mỗi bước bằng 1, cost cũng biểu diễn số bước của đường đi dẫn đến nút. Phương thức get_path_from_root() lần lượt đi theo các tham chiếu parent rồi đảo ngược kết quả để khôi phục dãy nút từ gốc đến đích.

Cách biểu diễn nút tách biệt dữ liệu môi trường khỏi thông tin tìm kiếm. Bản đồ mê cung chỉ được sử dụng để kiểm tra ô trống, tường và vị trí bắt đầu/đích; các quan hệ cha – con, chi phí và trạng thái đã khám phá được quản lý bởi đối tượng Node cùng các cấu trúc dữ liệu riêng. Khi trực quan hóa, chương trình tạo bản sao của mảng mê cung rồi mới đánh dấu vùng được mở rộng và đường đi, do đó không làm thay đổi dữ liệu gốc.

8.2.2. Tổ chức cây tìm kiếm, frontier và reached

Cây tìm kiếm được tạo động từ nút gốc và mở rộng theo các hành động hợp lệ. Frontier chứa những nút đã được sinh ra nhưng chưa được xử lý để tạo trạng thái kế tiếp. Nội dung và thứ tự của frontier quyết định chiến lược tìm kiếm: BFS sử dụng hàng đợi; DFS duyệt theo nhánh bằng stack; GBFS và A* sử dụng hàng đợi ưu tiên.

Ở BFS, reached là từ điển ánh xạ vị trí đến Node đã được phát hiện. Một vị trí được ghi nhận khi sinh ra, nhờ đó không bị thêm nhiều lần vào hàng đợi. Trong GBFS và A*, reached cũng lưu nút đại diện cho trạng thái; riêng A* so sánh chi phí g mới với chi phí đã ghi nhận nhằm thay thế đường đi kém hơn khi phát hiện một phương

án có chi phí thấp hơn. Cấu trúc closed và việc bỏ qua những bản ghi ưu tiên đã lỗi thời giúp giảm các lần mở rộng không cần thiết.

Đối với DFS, notebook triển khai kiểm tra chu trình trên đường đi hiện tại thay vì sử dụng reached hoặc visited toàn cục. Chương trình duy trì các khung duyệt của nhánh đang hoạt động và tập on_path để ngăn một trạng thái quay lại chính nhánh đó. Việc cho phép một trạng thái xuất hiện trên nhánh khác làm tăng khả năng khảo sát trùng nhưng thể hiện rõ khác biệt giữa tìm kiếm trên cây theo nhánh và tìm kiếm trên đồ thị với một tập trạng thái đã thăm toàn cục. Trên mê cung có nhiều chu trình, phương pháp này cần giới hạn số nút mở rộng để tránh tiêu tốn tài nguyên quá lớn.

Ngoài frontier, chương trình ghi nhận số nút mở rộng, độ sâu lớn nhất, kích thước frontier lớn nhất, thời gian thực thi và chỉ số đại diện cho số Node/entry đang được quản lý trong bộ nhớ. Chỉ số bộ nhớ này là phép đo quy ước dựa trên đối tượng dữ liệu, không phải lượng RAM thực tế tính theo byte và cũng không hoàn toàn cùng ý nghĩa giữa mọi thuật toán.

8.2.3. Triển khai BFS, DFS, GBFS và A*

BFS lấy nút ở đầu hàng đợi theo nguyên tắc First-In, First-Out (FIFO), vì vậy các trạng thái ở độ sâu nhỏ được xử lý trước. Với chi phí bước bằng 1, BFS tìm được đường có số bước tối thiểu. Thuật toán sử dụng reached để mỗi vị trí chỉ được phát hiện lần đầu một lần, giúp tránh lặp vô ích trên đồ thị mê cung.

DFS sử dụng nguyên tắc ưu tiên đi sâu; trong bản cài đặt Ex1, quá trình được tổ chức bằng các khung duyệt mô phỏng đệ quy và tập on_path. DFS tiếp tục trên nhánh đang xét đến khi gặp đích, ngõ cụt hoặc giới hạn tìm kiếm, sau đó quay lui. Vì không khảo sát theo độ sâu tăng dần, đường đi đầu tiên của DFS không được bảo đảm tối ưu. Thứ tự duyệt hướng N, E, S, W có thể ảnh hưởng lớn đến số nút mở rộng và độ dài lời giải.

GBFS lựa chọn nút theo heuristic h(n), được sử dụng trong notebook với khoảng cách Manhattan đến đích. Cách lựa chọn này ưu tiên vị trí được ước lượng gần đích hơn mà không trực tiếp xét chi phí đã đi. Vì vậy, GBFS có thể mở rộng ít nút nhưng

vẫn trả về một đường đi dài khi vật cản làm cho hướng tiến gần đích không trùng với đường đi ngắn nhất.

A* sử dụng hàm đánh giá f(n) = g(n) + h(n). Thành phần g(n) lưu chi phí đã tích lũy, còn h(n) ước lượng chi phí còn lại. Trong môi trường bốn hướng với chi phí mỗi bước bằng 1, khoảng cách Manhattan là heuristic admissible và consistent; cùng với cơ chế quản lý trạng thái, cập nhật chi phí phù hợp, A* tìm được đường đi tối ưu. Trong notebook, kết quả A* trên cả tám mê cung được đối chiếu tự động với kết quả BFS để kiểm tra độ dài đường đi.

8.3. Phần thực hành nâng cao

8.3.1. Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) thực hiện nhiều lượt Depth-Limited DFS (DLS) với giới hạn độ sâu lần lượt tăng từ 0, 1, 2, ... cho đến khi gặp đích hoặc đạt giới hạn tài nguyên. IDS tìm kiếm theo độ sâu tăng dần như BFS, nhưng mỗi lượt chỉ cần quản lý cấu trúc tìm kiếm theo nhánh của DFS. Việc lặp lại từ độ sâu nhỏ giúp tìm được nghiệm nông nhất khi các lượt DLS được triển khai đúng và đủ tài nguyên, đồng thời phát sinh chi phí mở rộng lại nhiều nút ở những lần tìm kiếm liên tiếp.

Bản cài đặt trong Ex1 thực hiện DLS bằng cơ chế kiểm tra chu trình theo đường đi hiện tại; giới hạn được đặt theo số nút mở rộng ở từng lượt và toàn bộ quá trình. Khi đạt giới hạn, chương trình ghi LIMIT_REACHED thay vì kết luận vô nghiệm. Cách triển khai này giữ notebook có thể chạy ổn định, nhưng đồng thời khiến một số mê cung chưa thể tìm được lời giải trong phạm vi thực nghiệm.

Bảng 8.1. Kết quả khảo sát IDS trên tám mê cung

Mê cung	Trạng thái	Cost	Lượt DLS	Tăng L	Nút mở rộng	Max frontier
small	Tìm thấy	19	20	19	1.051	20
medium	Giới hạn	—	63	62	12.000	63
large	Giới hạn	—	95	94	12.000	94
open	Giới hạn	—	10	9	5.555	10
L	Giới hạn	—	8	7	4.828	8
loops	Giới hạn	—	17	16	8.326	17
empty	Giới hạn	—	9	8	5.060	9

empty_maze_2	Giới hạn	—	9	8	5.060	9

Ghi chú: Dấu "—" biểu thị chưa tìm được lời giải trước giới hạn của lượt chạy.

Trên small_maze, IDS thử các mức giới hạn từ 0 đến 19, tổng cộng 20 lượt DLS và 19 lần tăng giới hạn, tìm được lời giải 19 bước sau 1.051 nút mở rộng cộng dồn. Kết quả cùng chi phí tối ưu với BFS, nhưng số nút mở rộng lớn hơn do những tầng nông được khảo sát lại. Các mê cung còn lại trong bảng không tìm thấy đích trước giới hạn tài nguyên của phiên bản thực nghiệm; những ô chi phí trống không biểu thị rằng mê cung không có lời giải.

8.3.2. Bài toán tìm kiếm nhiều đích

Trong phần nhiều đích, tác tử không phải đạt một tọa độ G duy nhất mà chỉ cần đến một trong các phần tử của tập $G = \{g_1, g_2, ..., g\square\}$. Điều kiện đạt mục tiêu vì vậy được thay bằng kiểm tra xem trạng thái hiện tại có thuộc tập G hay không. Đối với phương pháp có heuristic, ước lượng đến tập đích có thể được lấy là giá trị nhỏ nhất trong các khoảng cách đến từng đích.

Thực nghiệm được thực hiện trên bản sao của medium_maze, bổ sung hai vị trí đích vào bản đồ đã có sẵn một đích ban đầu. Ba tọa độ đích trong lần chạy là (1, 9), (1, 30) và (16, 1), theo chỉ số hàng–cột bắt đầu từ 0. Mê cung gốc không bị sửa đổi; các thay đổi chỉ được áp dụng trên bản sao tạo ra cho thí nghiệm.

Bảng 8.2. Kết quả tìm kiếm nhiều đích trên medium_maze

Thuật toán	Trạng thái	Path cost	Nút mở rộng	Max frontier
BFS	Tìm thấy	4	8	2
DFS	Tìm thấy	84	84	84
IDS	Tìm thấy	4	24	5

Ghi chú: Dữ liệu: ba đích trên bản sao medium_maze, mỗi bước có chi phí 1.

BFS và IDS cùng tìm được đích sau 4 bước, là số bước tối thiểu đến đích gần nhất trong thiết lập này; DFS tìm được một lời giải dài 84 bước. Kết quả một lần nữa cho thấy tính chất không tối ưu của DFS khi ưu tiên đi sâu trước. Đồng thời, việc có nhiều trạng thái đích có thể giúp rút ngắn độ sâu lời giải nếu xuất hiện một đích ở gần trạng thái bắt đầu hơn đích ban đầu.

8.4. Tổng hợp kết quả thực nghiệm

8.4.1. Môi trường và điều kiện thực nghiệm

Kết quả được tổng hợp từ Ex1_Maze.ipynb và Ex2_Maze_simplified.ipynb, sử dụng dữ liệu small, medium, large, open, L, loops, empty và empty_maze_2. Các thuật toán chạy trên bản đồ cùng mô hình di chuyển bốn hướng N, E, S, W với chi phí mỗi bước bằng 1. Hàm lập kế hoạch phải trả về chuỗi hành động hợp lệ và không được ghi trực tiếp frontier, reached hoặc đường đi vào dữ liệu mê cung trong quá trình tìm kiếm.

Bốn thuật toán BFS, DFS, GBFS và A* được kiểm thử trên toàn bộ tám mê cung, tạo thành 32 trường hợp. DFS có giới hạn 20.000 nút mở rộng để tránh kéo dài thời gian trên những bản đồ nhiều chu trình. Thời gian chạy được đo bằng bộ đếm hiệu năng của Python rồi quy đổi sang mili giây; đây là giá trị của lượt thực thi được ghi nhận, chưa phải trung bình từ nhiều lần chạy nên chỉ có ý nghĩa tham khảo.

Các chỉ số được sử dụng gồm: Status (kết quả FOUND, NO_PATH hoặc LIMIT_REACHED), Path cost (tổng chi phí đường đi), Nodes expanded (số nút được mở rộng), Max tree depth (độ sâu lớn nhất được ghi nhận theo cách cài đặt), Max frontier (kích thước frontier lớn nhất) và Time (ms) (thời gian chạy). Với BFS, Max tree depth có thể là độ sâu của nút được sinh ra dù đích nằm ở tầng nông hơn; với DFS, chỉ số này có thể lớn hơn chiều dài đường đi cuối cùng do thuật toán đã đi sâu ở nhánh khác trước đó.

8.4.2. Bảng so sánh các thuật toán trên các mê cung

Bảng 8.3. Chi phí đường đi của bốn thuật toán trên tám mê cung

Mê cung	BFS	DFS	GBFS	A*
small	19	37	29	19
medium	68	244	74	68
large	210	210	210	210
open	54	Giới hạn	68	54
L	16	50	24	16
loops	23	41	35	23
empty	14	34	14	14
empty_maze_2	14	80	14	14

Ghi chú: Giá trị “Giới hạn” tại open/DFS nghĩa là đã dừng sau 20.000 lượt mở rộng.

Bảng 8.3 cho thấy BFS và A* luôn cho chi phí bằng nhau trên cả tám mê cung. Trong khi đó, DFS và GBFS có thể tìm đường dài hơn: trên medium_maze, BFS/A* tìm đường 68 bước, còn DFS tìm đường 244 bước và GBFS tìm đường 74 bước. Trên open_maze, DFS chưa tìm được đích trong 20.000 lượt mở rộng; BFS/A* vẫn trả về lời giải 54 bước, còn GBFS tìm đường 68 bước. Ngoại lệ như large_maze, nơi cả bốn thuật toán đều trả về 210 bước, chỉ cho thấy chúng tìm được cùng chi phí trong bản đồ cụ thể, không tạo ra bảo đảm tối ưu chung cho DFS hay GBFS.

Bảng 8.4. Thống kê chi tiết 32 lượt tìm kiếm

Mê cung	Thuật toán	Trạng thái	Cost	Expanded	Max depth	Frontier	Time (ms)
small	BFS	Tìm thấy	19	92	20	9	0,299
small	DFS	Tìm thấy	37	93	44	45	0,288
small	GBFS	Tìm thấy	29	39	29	5	0,184
small	A*	Tìm thấy	19	49	19	8	0,335
medium	BFS	Tìm thấy	68	269	69	9	1,118
medium	DFS	Tìm thấy	244	267	244	244	0,756
medium	GBFS	Tìm thấy	74	78	74	4	0,331
medium	A*	Tìm thấy	68	220	68	8	1,259
large	BFS	Tìm thấy	210	621	211	8	1,893
large	DFS	Tìm thấy	210	519	222	223	1,479
large	GBFS	Tìm thấy	210	464	210	22	2,149
large	A*	Tìm thấy	210	539	210	13	3,796
open	BFS	Tìm thấy	54	682	54	25	2,215
open	DFS	Giới hạn	—	20.000	588	589	77,050
open	GBFS	Tìm thấy	68	83	68	67	0,457
open	A*	Tìm thấy	54	159	54	150	1,315
L	BFS	Tìm thấy	16	151	17	17	0,490
L	DFS	Tìm thấy	50	50	50	50	0,115
L	GBFS	Tìm thấy	24	30	24	27	0,164
L	A*	Tìm thấy	16	37	16	25	0,309
loops	BFS	Tìm thấy	23	71	23	8	0,239
loops	DFS	Tìm thấy	41	54	41	41	0,208
loops	GBFS	Tìm thấy	35	51	35	10	0,222
loops	A*	Tìm thấy	23	51	23	11	0,331
empty	BFS	Tìm thấy	14	95	15	12	0,300
empty	DFS	Tìm thấy	34	34	34	34	0,058
empty	GBFS	Tìm thấy	14	14	14	29	0,094
empty	A*	Tìm thấy	14	14	14	29	0,151
empty_maze_2	BFS	Tìm thấy	14	95	15	11	0,315
empty_maze_2	DFS	Tìm thấy	80	80	80	80	0,162
empty_maze_2	GBFS	Tìm thấy	14	14	14	29	0,095
empty_maze_2	A*	Tìm thấy	14	14	14	29	0,151

Ghi chú: Max depth, Max frontier và thời gian được đo theo quy ước của notebook Ex1; thời gian chỉ là một lượt chạy.

Bảng 8.4 ghi đầy đủ những chỉ số chính phục vụ so sánh kỹ thuật. Trên small_maze, A* chỉ mở rộng 49 nút so với 92 nút của BFS, trong khi cả hai đều tìm lời giải 19 bước. Trên open_maze, A* mở rộng 159 nút so với 682 nút của BFS, cho thấy heuristic có thể thu hẹp đáng kể số nút cần xử lý. Tuy nhiên, Max frontier của A* trên open_maze đạt 150, lớn hơn 25 của BFS; do đó, không thể suy ra ít nút mở rộng hơn đồng nghĩa ít bộ nhớ hơn.

Trong Ex2_Maze_simplified.ipynb, BFS được chạy độc lập trên bốn mê cung để kiểm tra tính nhất quán của phần triển khai rút gọn. Các đường đi và số nút mở rộng lần lượt trùng với kết quả BFS trong Ex1; thời gian thực thi có chênh lệch nhỏ giữa hai lần chạy.

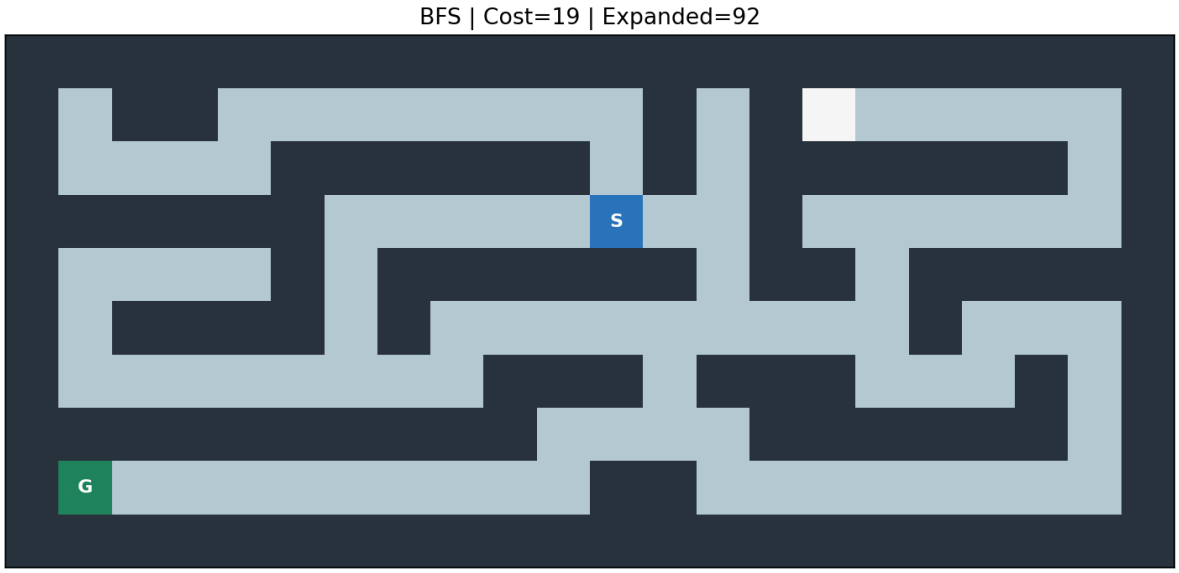
Bảng 8.5. Kết quả kiểm tra BFS trong notebook Ex2 rút gọn

Mê cung	Trạng thái	Path cost	Expanded	Max depth	Frontier	Time (ms)
small	Tìm thấy	19	92	20	9	0,341
large	Tìm thấy	210	621	211	8	1,841
open	Tìm thấy	54	682	54	25	2,334
empty	Tìm thấy	14	95	15	12	0,323

Ghi chú: Các kết quả đường đi và số nút mở rộng khớp BFS trong Ex1.

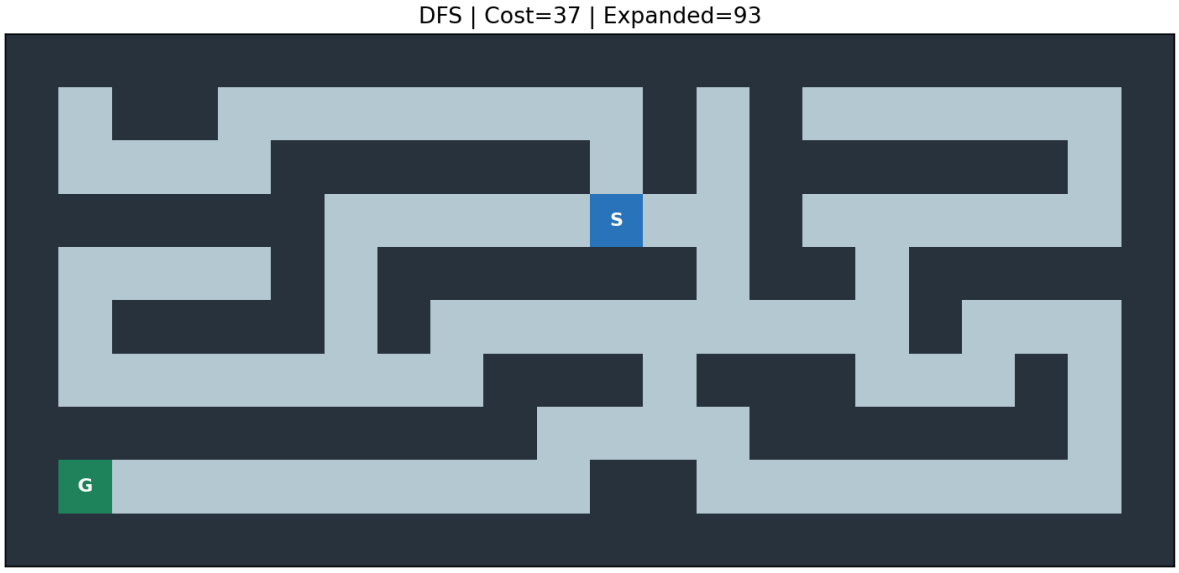
8.4.3. Trực quan hóa đường đi và trạng thái tìm kiếm

Notebook sinh các hình minh họa trạng thái được mở rộng và đường đi tìm được trên small_maze. Đối với các hình này, các vùng màu xám xanh thể hiện những ô đã được khảo sát, ký hiệu S và G chỉ trạng thái đầu và đích. Vì cùng một bản đồ có thể được vẽ từ nhiều kết quả tìm kiếm, quá trình đánh dấu được thực hiện trên bản sao của dữ liệu, tránh ảnh hưởng đến các lần chạy sau.



Hình 8.1. Vùng trạng thái được mở rộng bởi BFS trên small_maze.

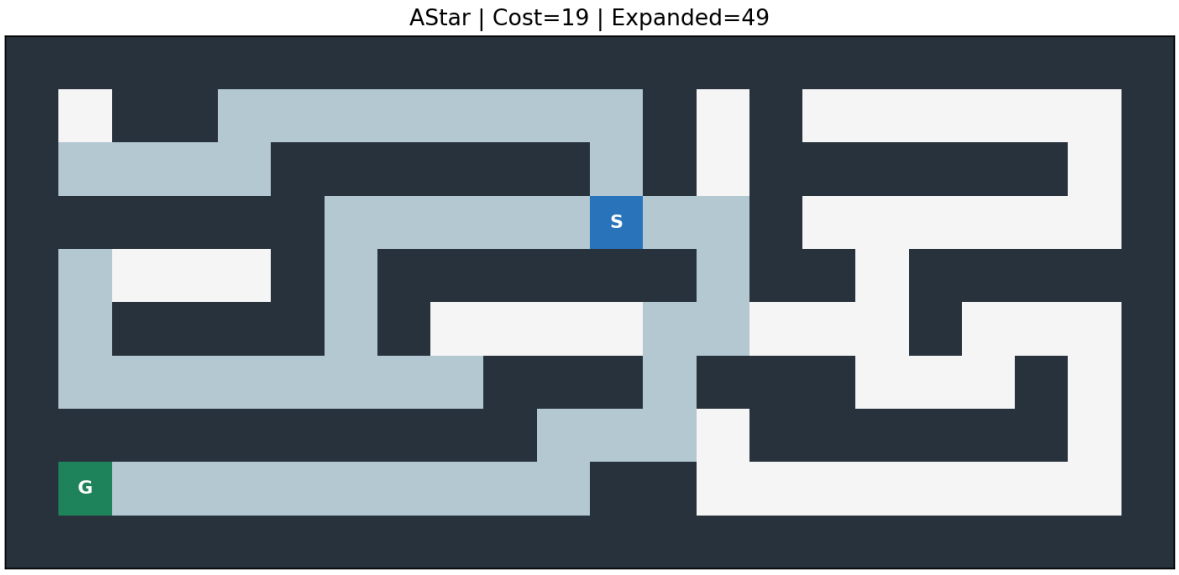
Hình 8.1 cho thấy BFS khảo sát nhiều vị trí trong mê cung và tìm được đường 19 bước. Vì BFS không sử dụng heuristic, việc mở rộng diễn ra theo độ sâu tăng dần từ S, bao phủ nhiều ô không nằm trên đường đi cuối cùng.



Hình 8.2. Vùng trạng thái được mở rộng bởi DFS trên small_maze.

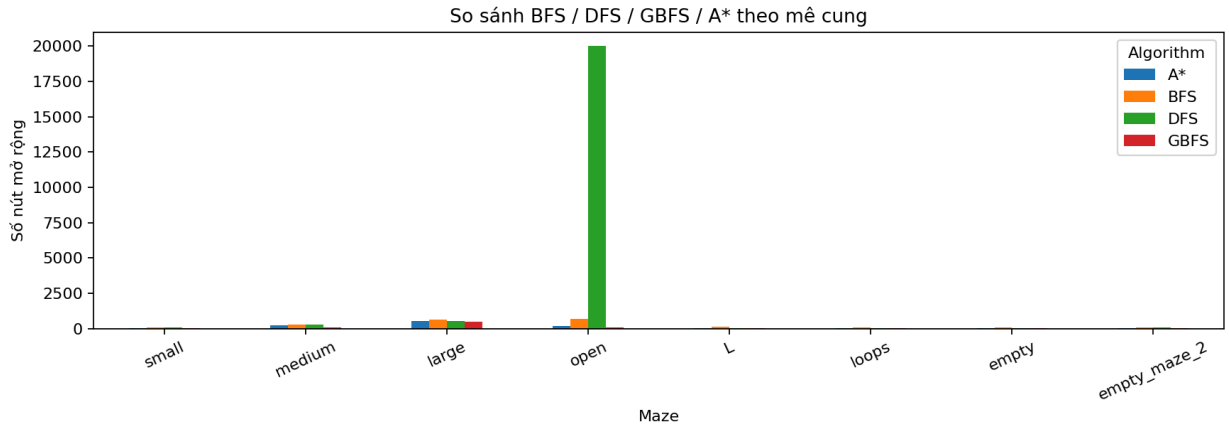
Hình 8.2 thể hiện DFS tìm được đường 37 bước và mở rộng 93 nút trên small_maze. Mặc dù hai hình BFS và DFS đều có vùng đã khảo sát khá rộng, DFS

có thể đi sâu theo một nhánh chưa tối ưu và quay lui trước khi gặp đích. Đường cuối cùng dài hơn BFS 18 bước.



Hình 8.3. Vùng trạng thái được mở rộng bởi A* trên small_maze.

Hình 8.3 thể hiện A* mở rộng 49 nút và tìm lời giải 19 bước. Vùng đã khảo sát tập trung hơn so với BFS, phù hợp với việc sử dụng chi phí tích lũy kết hợp khoảng cách Manhattan để định hướng tìm kiếm.



Hình 8.4. So sánh số nút mở rộng của BFS, DFS, GBFS và A* trên tám mê cung.

Biểu đồ tổng hợp số nút mở rộng cho thấy sự khác nhau giữa các thuật toán trên tám mê cung. Đặc biệt, cột DFS tại open_maze đạt 20.000 do chạm giới hạn thực

nghiệm nên cần được đọc là khối lượng công việc đến thời điểm dừng, không phải chi phí tìm được lời giải. Do giá trị này lớn hơn hẳn các trường hợp khác, biểu đồ có độ chênh lệch thang đo rất lớn; việc kết luận nên đối chiếu thêm với bảng số liệu thay vì chỉ dựa vào chiều cao cột.

8.5. Đánh giá tổng hợp các thuật toán tìm kiếm

8.5.1. Tính đầy đủ và tính tối ưu

Kết quả thực nghiệm cho thấy BFS và A* đều tìm được lời giải có chi phí tối ưu trên cả tám mê cung. Trong mô hình di chuyển bốn hướng với chi phí mỗi bước bằng 1, BFS mở rộng trạng thái theo độ sâu tăng dần, nhờ đó bảo đảm tìm được đường đi có số bước nhỏ nhất. A* sử dụng heuristic Manhattan kết hợp với chi phí tích lũy g(n), đồng thời quản lý các trạng thái đã phát hiện để tìm được lời giải có chất lượng tương đương BFS.

Trong khi đó, DFS và GBFS không bảo đảm tính tối ưu. Trên medium_maze, DFS tìm được đường đi 244 bước, còn GBFS tìm được đường 74 bước, trong khi đường đi tối ưu chỉ dài 68 bước. Sự khác biệt xuất phát từ nguyên tắc lựa chọn nút: DFS ưu tiên khám phá sâu một nhánh, còn GBFS chỉ sử dụng khoảng cách ước lượng đến đích mà không trực tiếp xét chi phí đã thực hiện.

Về tính đầy đủ, cần phân biệt đặc tính lý thuyết của thuật toán với kết quả trong phạm vi tài nguyên thực nghiệm. Trên open_maze, DFS dừng sau 20.000 lượt mở rộng mà chưa tìm được đích. Kết quả này không chứng minh mê cung không có lời giải, bởi BFS và A* đã xác định được đường đi 54 bước. Do đó, trạng thái LIMIT_REACHED được sử dụng để phản ánh chính xác nguyên nhân dừng tìm kiếm.

8.5.2. Thời gian thực thi và nhu cầu bộ nhớ

Số nút mở rộng cho phép đánh giá lượng công việc tìm kiếm nhưng không phản ánh đầy đủ thời gian thực thi. Trên open_maze, BFS mở rộng 682 nút, trong khi A*

chỉ mở rộng 159 nút, tương ứng mức giảm khoảng 76,7%. Trên small_maze, A* cũng giảm số nút mở rộng từ 92 xuống 49 mà vẫn bảo đảm lời giải 19 bước.

Tuy nhiên, ít nút mở rộng hơn không đồng nghĩa với thời gian chạy luôn thấp hơn. Chẳng hạn, trên large_maze, A* mở rộng ít nút hơn BFS nhưng thời gian đo trong lượt chạy vẫn cao hơn do phải tính heuristic, xử lý hàng đợi ưu tiên và cập nhật thông tin chi phí. Các giá trị thời gian trong notebook được ghi nhận từ từng lượt thực thi, vì vậy chỉ có ý nghĩa tham khảo và chưa đủ để kết luận về tốc độ trung bình.

Về bộ nhớ, BFS cần quản lý queue và tập reached, còn A* sử dụng hàng đợi ưu tiên cùng các cấu trúc theo dõi chi phí. Trên open_maze, kích thước frontier lớn nhất của A* là 150, trong khi BFS chỉ đạt 25, mặc dù A* mở rộng ít nút hơn đáng kể. Điều này cho thấy số nút mở rộng và kích thước frontier là hai chỉ số khác nhau. Ngoài ra, số đối tượng Node được quản lý trong notebook chỉ là đại lượng đại diện, không phải lượng RAM thực tế.

8.5.3. Vai trò của heuristic

Heuristic đóng vai trò định hướng quá trình tìm kiếm bằng cách ước lượng chi phí còn lại từ trạng thái hiện tại đến đích. Trong GBFS, heuristic là thành phần duy nhất quyết định mức ưu tiên, vì vậy thuật toán có thể tiếp cận đích nhanh theo hướng ước lượng nhưng tìm được đường đi chưa tối ưu khi có vật cản.

Đối với A*, heuristic được kết hợp với chi phí đã thực hiện để cân bằng giữa tiến độ hướng đến đích và chất lượng đường đi. Kết quả trên small_maze và open_maze cho thấy A* sử dụng Manhattan có khả năng giảm đáng kể số nút mở rộng so với BFS mà vẫn giữ nguyên chi phí tối ưu.

Tuy nhiên, hiệu quả của heuristic còn phụ thuộc vào cấu trúc mê cung, thứ tự sinh trạng thái và quy tắc xử lý các nút có cùng giá trị ưu tiên. Do đó, việc sử dụng heuristic tốt không bảo đảm thuật toán luôn có thời gian hoặc nhu cầu bộ nhớ thấp nhất trong mọi trường hợp.

8.5.4. Ảnh hưởng của cycle checking

Kiểm tra chu trình là cơ chế quan trọng nhằm hạn chế việc thuật toán quay lại những trạng thái đã xuất hiện trong quá trình tìm kiếm. BFS sử dụng reached để tránh sinh lại cùng một vị trí, trong khi A* còn quản lý chi phí tốt nhất của trạng thái nhằm xử lý những đường đi có chất lượng khác nhau.

Đối với DFS trong notebook Ex1, việc kiểm tra chu trình chỉ được thực hiện trên đường đi hiện tại. Phương pháp này ngăn một nhánh tự quay lại trạng thái tổ tiên nhưng vẫn cho phép cùng một vị trí được khảo sát lại qua những nhánh khác nhau. Trên open_maze, hiện tượng khám phá lại trạng thái khiến DFS tiêu tốn nhiều lượt mở rộng và phải dừng ở giới hạn tài nguyên.

Phần IDS cũng chịu ảnh hưởng từ cơ chế kiểm tra chu trình của DLS. Mặc dù việc tăng dần giới hạn độ sâu giúp tìm kiếm lời giải nông nhất khi có đủ tài nguyên và triển khai đúng, thuật toán phải lặp lại quá trình duyệt các tầng trước đó. Điều này làm gia tăng tổng số nút mở rộng, đặc biệt trên những mê cung có nhiều nhánh và chu trình.

Nhìn chung, cycle checking cần được thiết kế phù hợp với từng chiến lược tìm kiếm. Một cơ chế quản lý trạng thái không phù hợp có thể làm tăng việc khám phá lại hoặc loại bỏ nhầm những đường đi hợp lệ. Vì vậy, tính hiệu quả của thuật toán phụ thuộc không chỉ vào chiến lược lựa chọn nút mà còn vào cách tổ chức frontier và reached.

8.6. Kết luận bài thực hành tổng hợp

Bài thực hành đã hoàn thiện quy trình xây dựng Goal-based Planning Agent từ khâu mô hình hóa mê cung, tổ chức cây tìm kiếm và dữ liệu Node đến cài đặt các hàm lập kế hoạch, truy hồi chuỗi hành động, kiểm tra tính hợp lệ và trực quan hóa kết quả. Các thuật toán BFS, DFS, GBFS và A* được triển khai trên cùng tám mê cung, tạo cơ sở so sánh giữa tìm kiếm không thông tin và tìm kiếm có thông tin theo các chỉ số chi phí đường đi, số nút mở rộng, độ sâu, kích thước frontier và thời gian thực thi.

Kết quả xác nhận BFS và A* tìm được lời giải tối ưu trong thiết lập chi phí bước đồng nhất. A* dùng Manhattan thường giảm số nút mở rộng so với BFS khi heuristic định hướng hiệu quả. DFS và GBFS có thể tìm đường nhanh theo một số chỉ số ở những trường hợp cụ thể nhưng không bảo đảm tối ưu; DFS còn chịu ảnh hưởng mạnh từ cơ chế kiểm tra chu trình và giới hạn tài nguyên. Thí nghiệm IDS cho thấy lợi ích của việc tìm kiếm theo độ sâu tăng dần cùng chi phí mở rộng lại các tầng trước đó, còn bài toán nhiều đích minh họa khả năng mở rộng điều kiện hoàn thành kế hoạch của tác tử.

Qua đó, bài thực hành nhấn mạnh rằng không tồn tại một chiến lược tìm kiếm tốt nhất trong mọi điều kiện đánh giá. Việc lựa chọn thuật toán cần dựa trên yêu cầu về tối ưu, thời gian, bộ nhớ, cấu trúc môi trường và mức độ tin cậy của heuristic. Những kiến thức này tạo nền tảng cho việc thiết kế các tác tử lập kế hoạch và hệ thống giải quyết vấn đề bằng tìm kiếm trên những không gian trạng thái phức tạp hơn.